

MODULE 2

SUPERVISED LEARNING NETWORK

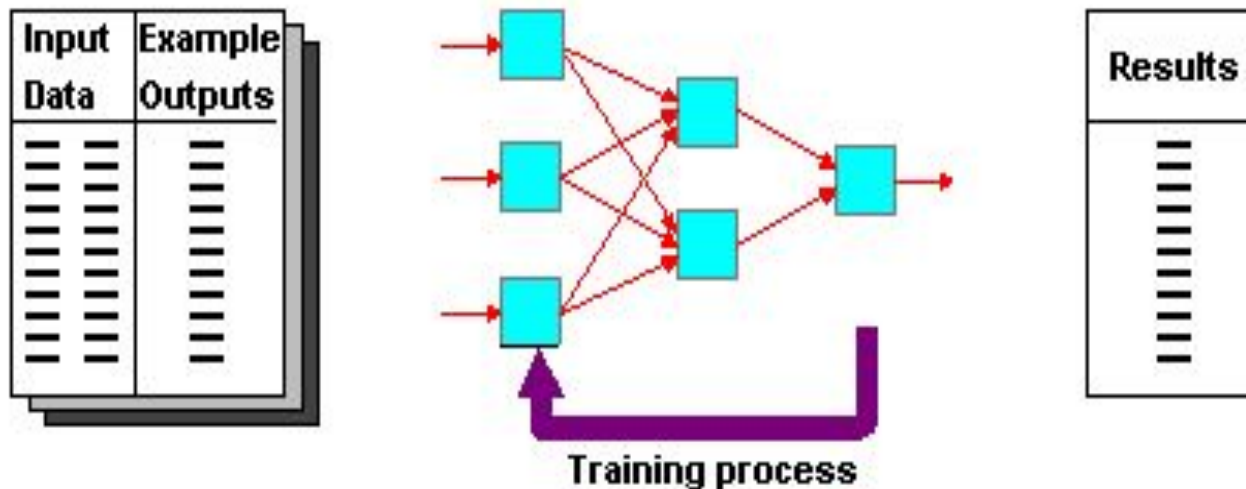
Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam

Module – 2 (Supervised Learning Network)

Perceptron Networks– Learning rule, Training and testing algorithm. Adaptive Linear Neuron– Architecture, Training and testing algorithm. Back propagation Network – Architecture, Training and testing algorithm.

DEFINITION OF SUPERVISED LEARNING NETWORKS

- Training and test data sets
- Training set; input & target are specified



LAYERS IN NEURAL NETWORK

□ The input layer:

- Introduces input values into the network.
- No activation function or other processing.

□ The hidden layer(s):

- Performs classification of features.
- Two hidden layers are sufficient to solve any problem.
- Features imply more layers may be better.

□ The output layer:

- Functionally is just like the hidden layers.
- Outputs are passed on to the world outside the neural network.

PERCEPTRON NETWORKS

- Single layer feed forward network
- First neural network learning model in the 1960's
- Simple and limited (single layer models)
- Basic concepts are similar for multi-layer models so this is a good learning tool
- Still used in many current applications (modems, etc.)
- Called Simple perceptron
- Discovered by Block in 1962

PERCEPTRON NETWORKS

- ✓ Key points to be noted in perceptron network are
- ❖ Consist of 3 units
 - *Sensory unit(input unit)*
 - *Associator Unit(hidden unit)*
 - *Response unit(output unit)*
 - ❖ *Sensory units* are connected to *associator unit* with a fixed weight having values 1, 0 or -1 which are assigned at random
 - ❖ Binary activation functions are used in the sensory unit and associator unit
 - ❖ The *response unit* has an activation of 1,0 or -1
 - ❖ The binary step with fixed threshold θ is used as an activation for associator
 - ❖ The output signal that are send from the associator to response are only binary

- ❖ The output of perceptron network is given by

$$Y = f(y_{in})$$

- ❖ Where $f(y_{in})$ is the activation function and is defined as

$$f(y_{in}) =$$

$$\begin{cases} 1 & \text{If } y_{in} > \theta \\ 0 & \text{If } -\theta \leq y_{in} \leq \theta \\ -1 & \text{If } y_{in} < -\theta \end{cases}$$

- ❖ The perceptron learning rule is used in the weight updation between the associator unit and the response unit
- ❖ For each training input, the net will calculate the response and it will determine whether or not an error has occurred
- ❖ The error calculations are based on the comparison of values for targets with those of calculated output

- ❖ The weight s on the connection from the unit that send the non zero signal will get adjusted suitably
- ❖ Weight will be adjusted on the basis of the learning rule if an error has occurred for a particular training pattern

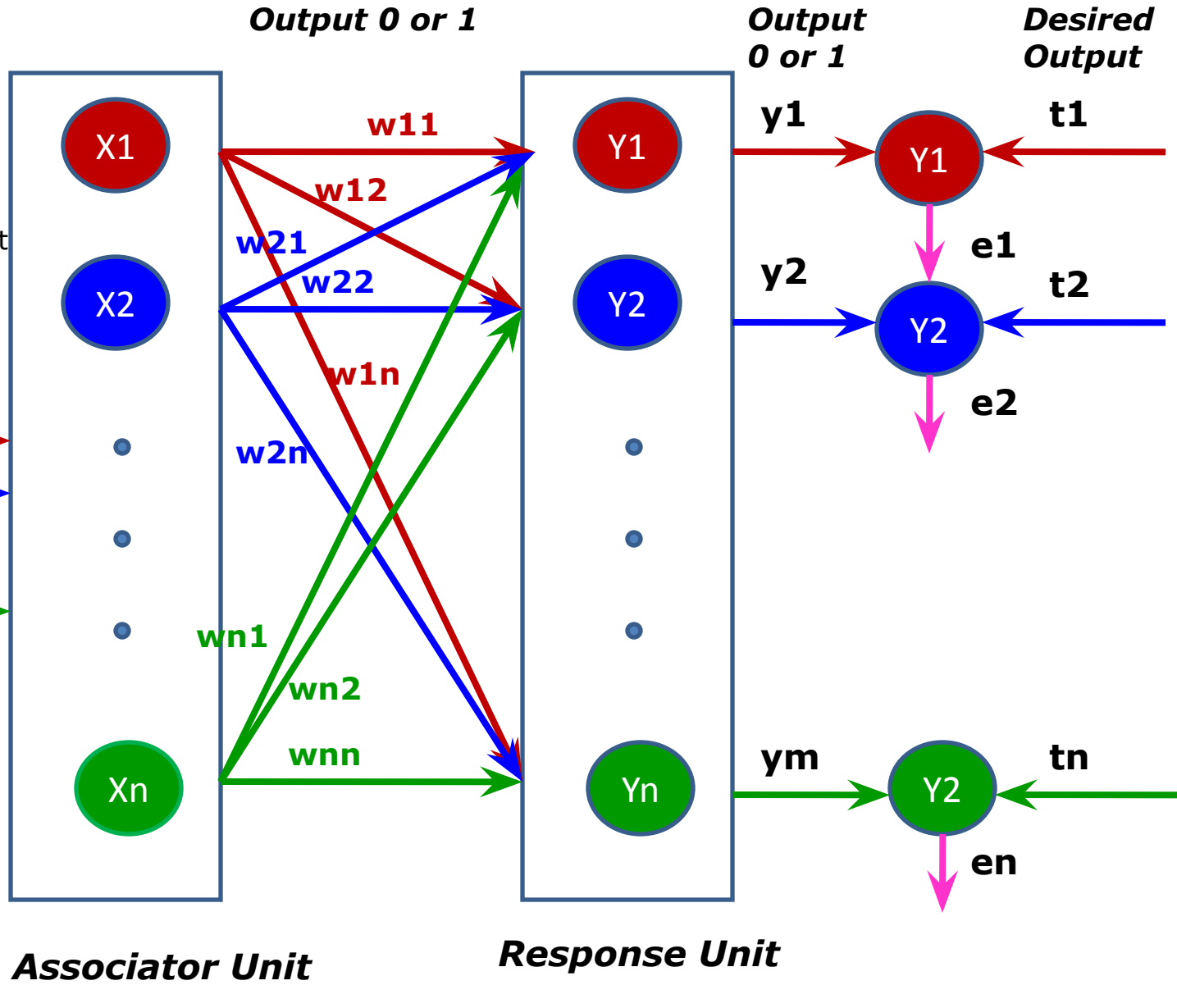
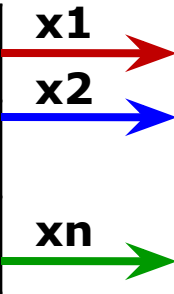
$$w_i(new) = w_i(old) + \alpha t x_i$$

$$b(new) = b(old) + \alpha t$$

- ❖ If no error occurred then no weight updation and hence the training process may be stopped
- ❖ In the above equation the target value 't' is +1 or -1
- ❖ α is the learning rate

X	X	X
	X	
	X	

Fixed weight
value of
0,1,-1 at
random



✓ **Sensory unit**

- can be a 2D matrix of 400 photo detectors
- These detectors provide a binary electrical signal if the input signal is found to exceed a certain value of threshold
- Also these detectors are connected randomly with the associator unit

✓ **Associator unit**

- consist of a set of sub circuits called **feature predicates**
 - Hardwired to detect the specific feature of a pattern
- For a particular feature each predicate is examined with a few or all of the responses of sensory unit
- The result from these predicate unit is also binary

✓ **Response Unit**

- Contain pattern recognizer or perceptron
- The weight present in the input layer are all fixed
- While weight in the response layer are trainable

LEARNING ALGORITHM

- **Epoch** : Presentation of the entire training set to the neural network.
- In the case of the AND function, an epoch consists of four sets of inputs being presented to the network (i.e. [0,0], [0,1], [1,0], [1,1]).
- **Error**: The error value is the amount by which the value output by the network differs from the target value. For example, if we required the network to output 0 and it outputs 1, then Error = -1.

PERCEPTRON LEARNING RULE

- ✓ In learning rule, the learning signal is the difference between the desired and actual response of the neuron
- ✓ Learning rule explained as follows
 - Consider a finite 'n' number of input training vectors, with their associated target values $x(n)$ and $t(n)$, where n ranges from 1 to n
 - The target is either +1 or -1
 - The output y is obtained on the basis of the net input calculated and activation function being applied over the net input

$$y = f(y_{in}) =$$

$$\begin{cases} 1 & \text{If } y_{in} > \theta \\ 0 & \text{If } -\theta \leq y_{in} \leq \theta \\ -1 & \text{If } y_{in} < -\theta \end{cases}$$

- Weight updation in case of perceptron learning is as shown
If $y \neq t$ then

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \alpha t \mathbf{x} \quad (\alpha \text{ learning rate})$$

Else we have

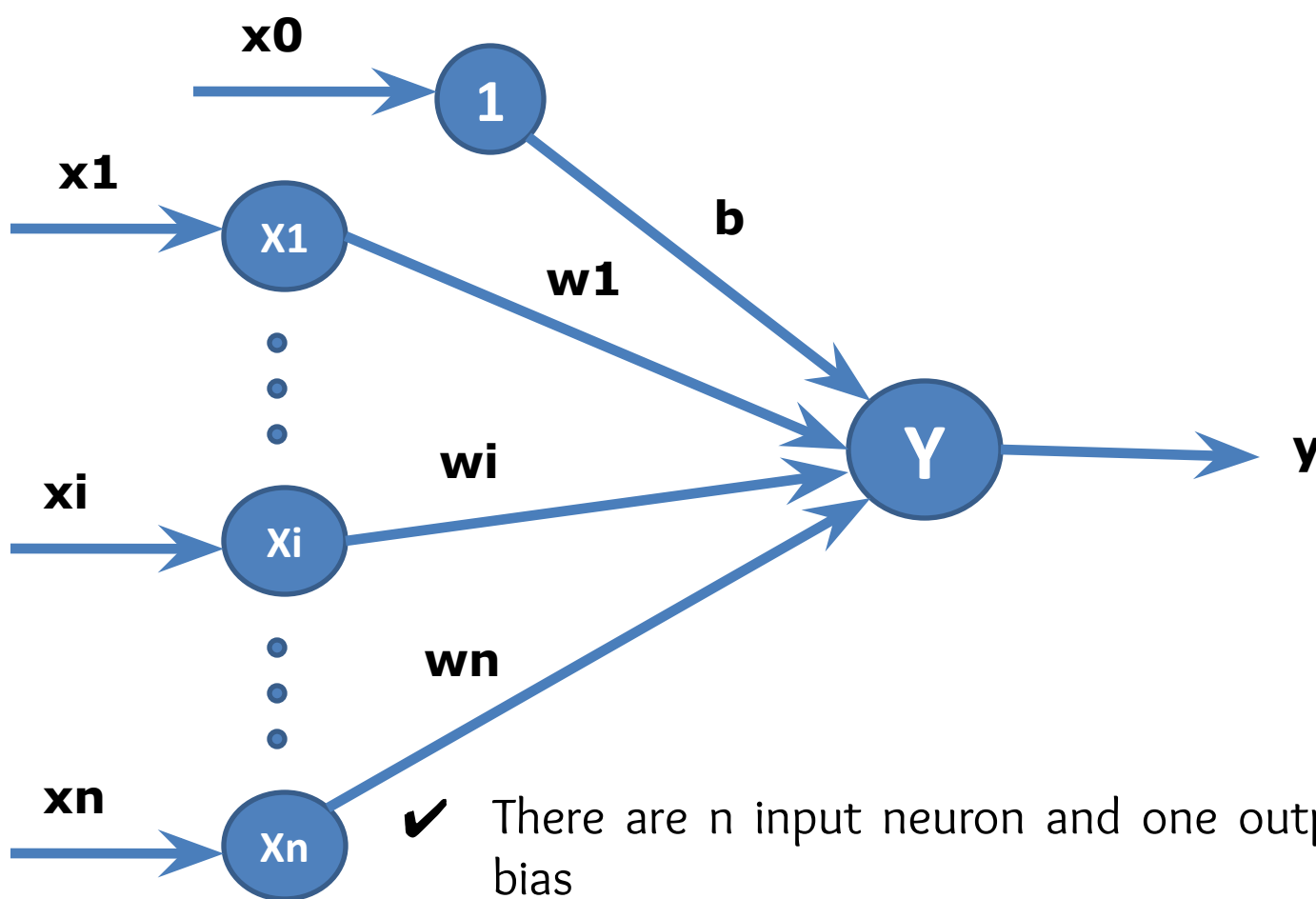
$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old})$$

- Weight can be initialized to any values in this method
- Perceptron rule convergence theorem states that

”If there is a weight vector $\mathbf{W}$, such that $f(\mathbf{x}(n) \cdot \mathbf{W}) = t(n)$, for all n , then for any starting vector $\mathbf{W}_1$, the perceptron learning rule converges to a weight vector that gives the correct response for all training patterns and this learning take place within a finite number of steps provided that the solution exist”

ARCHITECTURE

- ✓ The output obtained from the associator unit is a binary vector and hence that the output is taken as input signal to the response unit
- ✓ Here the weight between the sensory unit and associator unit are fixed
- ✓ Weight between the associator unit and response unit output unit can be adjusted
- ✓ As a result the discussion of the network is limited to a single portion
- ✓ Thus associator unit behaves like input unit



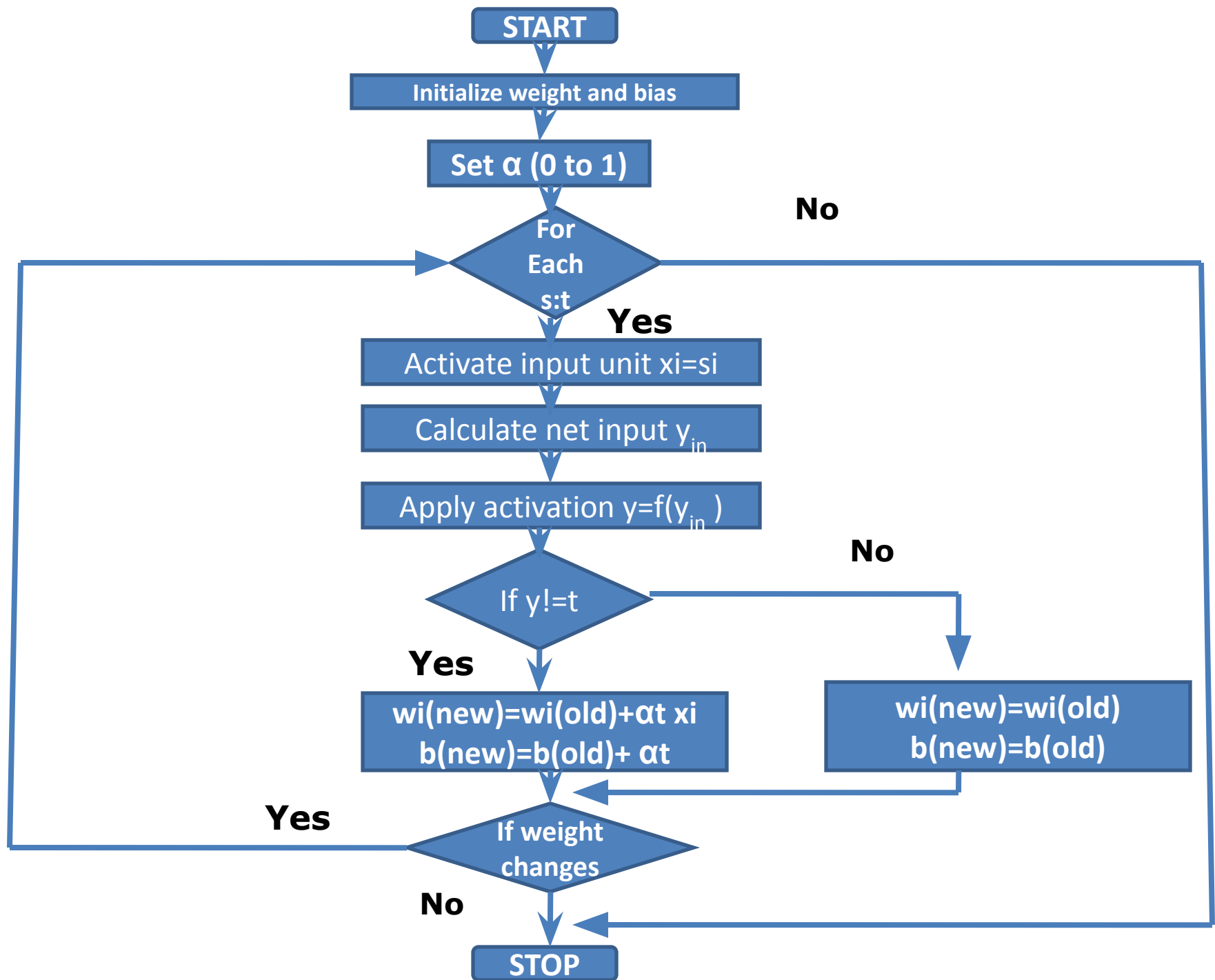
- ✓ There are n input neuron and one output neuron and a bias
- ✓ The input layer and output layer are connected through the direct communication link which is associated with weight
- ✓ The goal of perceptron network is to classify the input pattern as the member or not a member of a particular class

TRAINING ALGORITHM

- Adjust neural network weights to map inputs to outputs.
- Use a set of sample patterns where the desired output (given the inputs presented) is known.
- The purpose is to learn to
 - Recognize features which are common to good and bad

FLOW CHART OF A TRAINING PROCESS

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam



PERCEPTRON TRAINING ALGORITHM FOR SINGLE OUTPUT CLASS

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam

Step 0: Initialize the weight and bias (to zero). Also initialize learning rate α ($0 < \alpha \leq 1$) For simplicity α set to 1

Step 1: Perform step 2- 6 until the final stopping condition is false

Step 2: Perform step 3-5 for each training pair indicated by s:t

Step 3: Input layer containing the input unit is applied with identity activation functions

$$x_i = s_i$$

Step 4: Calculate the output of the network. To do so first obtain the net input

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

'n' is the number of neurons in the layer

Then apply the activation function over net input to get the output

$$y = f(y_{in}) = \begin{cases} 1 & \text{If } y_{in} > \theta \\ 0 & \text{If } -\theta \leq y_{in} \leq \theta \\ -1 & \text{If } y_{in} < -\theta \end{cases}$$

Step 5: Weight and bias adjustment: Compare values of actual and desired output

If $y \neq t$ then $w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$

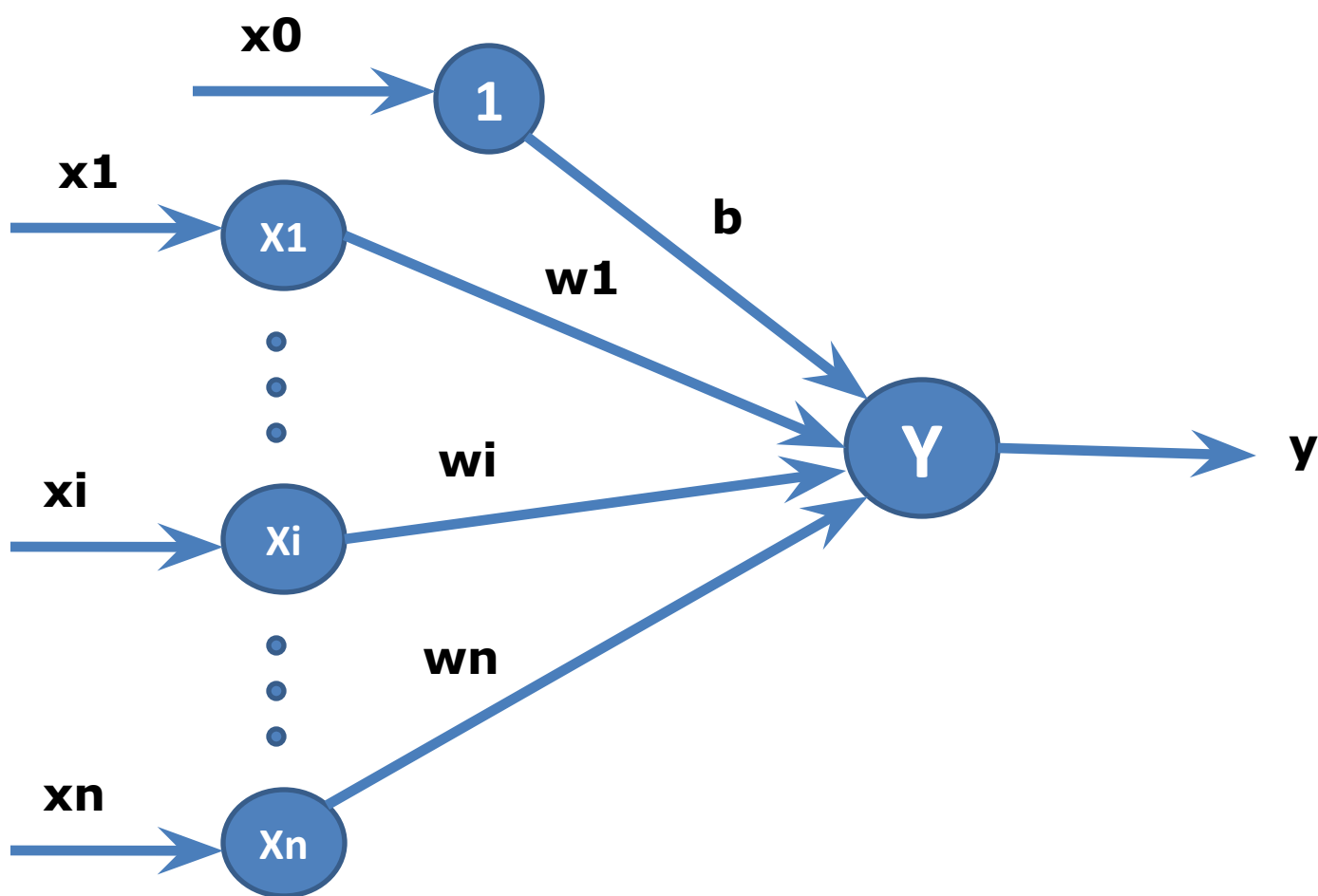
$$b(\text{new}) = b(\text{old}) + \alpha t$$

else $w_i(\text{new}) = w_i(\text{old})$

$$b(\text{new}) = b(\text{old})$$

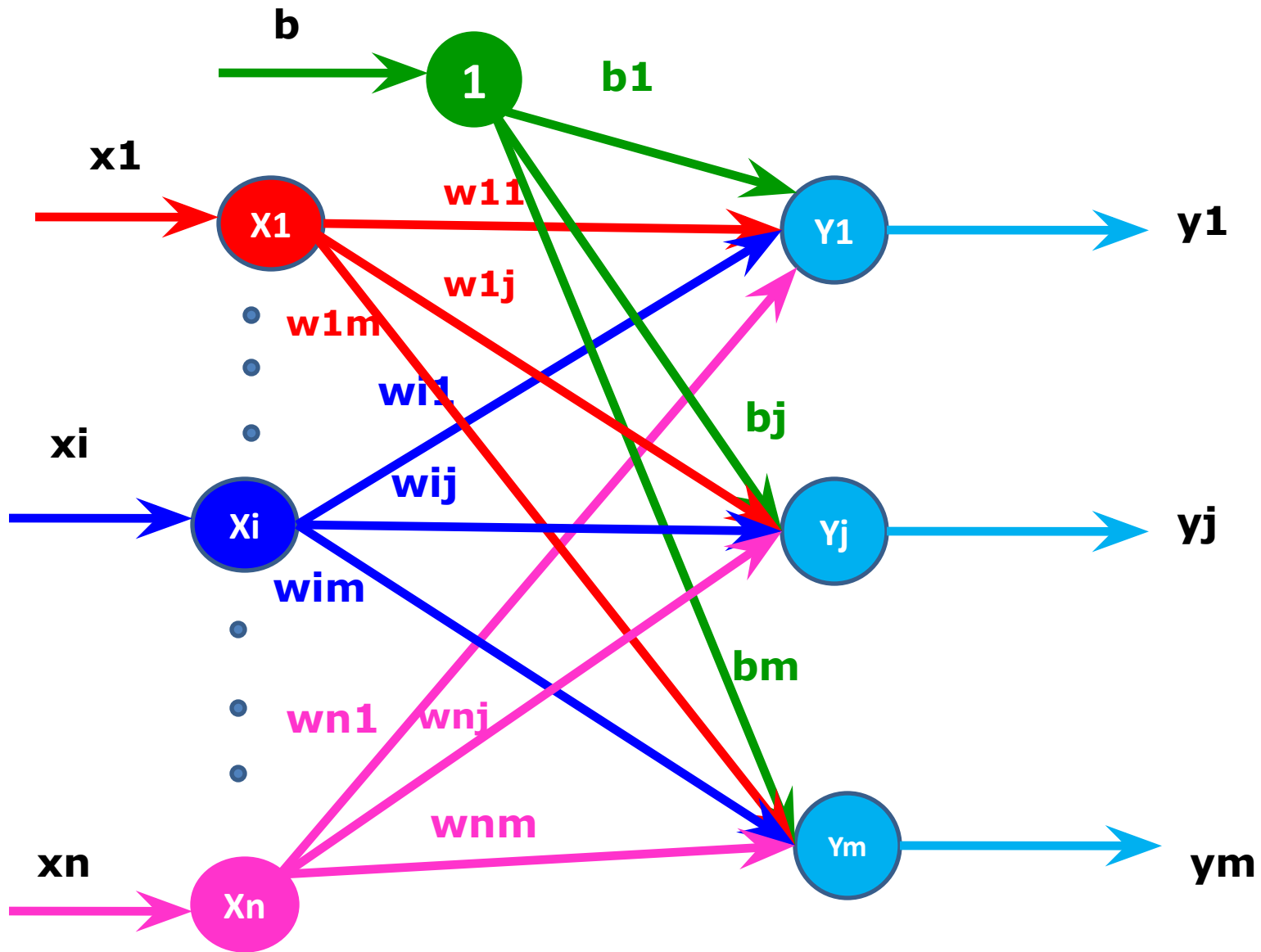
Step 6: Train the network until there is no weight change. This is the stopping condition for the network. If this condition is not met, then start again from step 2

- Notes:
 - Learning occurs only when a sample has $y \neq t$
 - Otherwise, it's similar to Hebb Learning rule
 - Two loops, a completion of the inner loop (each sample is used once) is called an epoch
- **Stop condition**
 - **When no weight is changed in the current epoch, or**
 - **When pre-determined number of epochs is reached**



PERCEPTRON TRAINING ALGORITHM FOR MULTIPLE OUTPUT CLASS

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam



Step 0: Initialize the weight and bias (to zero). Also initialize learning rate α ($0 < \alpha \leq 1$) For simplicity α set to 1

Step 1: Perform step 2- 6 until the final stopping condition is false

Step 2: Perform step 3-5 for each training pair indicated by s:t

Step 3: Set the activation of each input unit $i=1$ to n

$$x_i = s_i$$

Step 4: Calculate the output response of each output from $j=1$ to m . To do so first obtain the net input

$$y_{in\ j} = b_j + \sum_{i=1}^n x_i w_{ij}$$

'n' is the number of neurons in the layer

Then apply the activation function over net input to get the output

$$y_j = f(y_{in\ j}) =$$

$$\begin{cases} 1 & \text{If } y_{in} > \theta \\ 0 & \text{If } -\theta \leq y_{in} \leq \theta \\ -1 & \text{If } y_{in} < -\theta \end{cases}$$

Step 5: Weight and bias adjustment: from $j=1$ to m and $i=1$ to n

If $y_j \neq t_j$ then $w_{ij}(new) = w_{ij}(old) + \alpha t_j x_i$

$$b_j(new) = b_j(old) + \alpha t_j$$

else $w_{ij}(new) = w_{ij}(old)$

$$b_j(new) = b_j(old)$$

Step 6: Train the network until there is no weight change. This is the stopping condition for the network. If this condition is not met, then start again from step 2

PERCEPTRON NETWORK TESTING ALGORITHM

Step 0: The initial weight used here are taken from the training algorithms (the final weight obtained during training)

Step 1: For each input vector X to be classified perform step 2 – 3

Step 2: Set activation of the input unit

Step 3: Obtain the response of the output unit

$$y_{in} = \sum x_i w_i$$
$$y = f(y_{in}) = \begin{cases} 1 & \text{If } y_{in} > \theta \\ 0 & \text{If } -\theta \leq y_{in} \leq \theta \\ -1 & \text{If } y_{in} < -\theta \end{cases}$$

- ✓ In case of perceptron network , it can be used for linear seperability concept
- ✓ Here the separating line may be based on the values of threshold
- ✓ The condition for separating the response from region of positive to region of zero is

$$w_1x_1 + w_2x_2 + b > \theta$$

- ✓ The condition for separating the response from region of zero to region of negative is

$$w_1x_1 + w_2x_2 + b < -\theta$$

- ✓ The condition above are stated for a single layer perceptron network with two input neuron and one output neuron and one bias

Implement AND function using perceptron network for bipolar input and targets

Implement OR function using perceptron network for binary input and bipolar targets up to 3 epochs

Find the weight using perceptron network for ANDNOT function when all the input are presented only one time. Use bipolar input and targets

Find the weight using perceptron network for ANDNOT function when all the input are presented only one time. Use bipolar input and targets

Find the weight required to perform the following classification using perceptron network. The vectors $(1,1,1,1)$ and $(-1,1,-1,-1)$ are belongs to the class so have the target value 1, vectors $(1,1,1,-1)$ and $(1,-1,-1,1)$ are not belongs to the class so have the target value -1. Assume learning rate 1 and initial weights as 0

Using Perceptron rule, Find the weight required to perform the following classification of the given input patterns shown below

The pattern is shown as 3X3 matrix form in the squares. The “+” symbol represents the value 1 and empty square indicate -1.

Consider “I” belongs to the member of the class so the target value is 1 and “F” does not belongs to the member of the class so the target value is -1

+	+	+
	+	
+	+	+

“I”

+	+	+
+	+	+
+		

“F”

TUTORIAL QUESTIONS

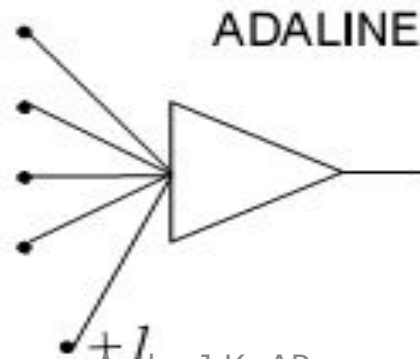
1. Implement NOR function using perceptron network for bipolar input and targets
2. Find the weight required to perform the following classification using perceptron network. The vectors $(1,1,-1,-1)$ and $(1,-1,1,-1)$ are belongs to the class so have the target value 1, vectors $(-1,-1,-1,1)$ and $(-1,-1,1,1)$ are not belongs to the class so have the target value -1. Assume learning rate 1 and initial weights as 0
3. Classify the 2D pattern shown in the figure below using perceptron network

+	+	+
+		
+	+	+

	+	
+	+	+
+	+	+

ADAPTIVE LINEAR NEURON (ADALINE)

In 1959, Bernard Widrow and Marcian Hoff of Stanford developed models they called ADALINE (Adaptive Linear Neuron) and MADALINE (Multilayer ADALINE). These models were named for their use of Multiple ADaptive LINear Elements. MADALINE was the first neural network to be applied to a real world problem. It is an adaptive filter which eliminates echoes on phone lines.



ADAPTIVE LINEAR NEURON (ADALINE)

- ✓ The units with linear activation functions are called linear units
- ✓ A network with a single linear unit is called an Adaline
- ✓ ie, In Adaline input output relationship is linear
- ✓ Adaline uses the bipolar activation for its input signal and its target output
- ✓ The weight between the input and output are adjustable
- ✓ The bias in Adaline act like an adjustable weight whose connection is from a unit with activation being always 1
- ✓ Adaline is a net which has only one output unit
- ✓ Trained using delta rule
- ✓ This rule is found to minimize the mean squared error between the activation and target value

ADALINE LEARNING RULE

Adaline network uses Delta Learning Rule. This rule is also called as Widrow-Hoff Learning Rule or Least Mean Square Rule. The delta rule for adjusting the weights is given as ($i = 1$ to n):

$$\Delta w_i = \alpha(t - y_{in})x_i$$

Δw_i = weight change

α = learning rate

x = vector of activation of input unit

y_{in} = net input to output unit, *i.e.*, $Y = \sum_{i=1}^n x_i w_i$

t = target output

ADALINE LEARNING RULE

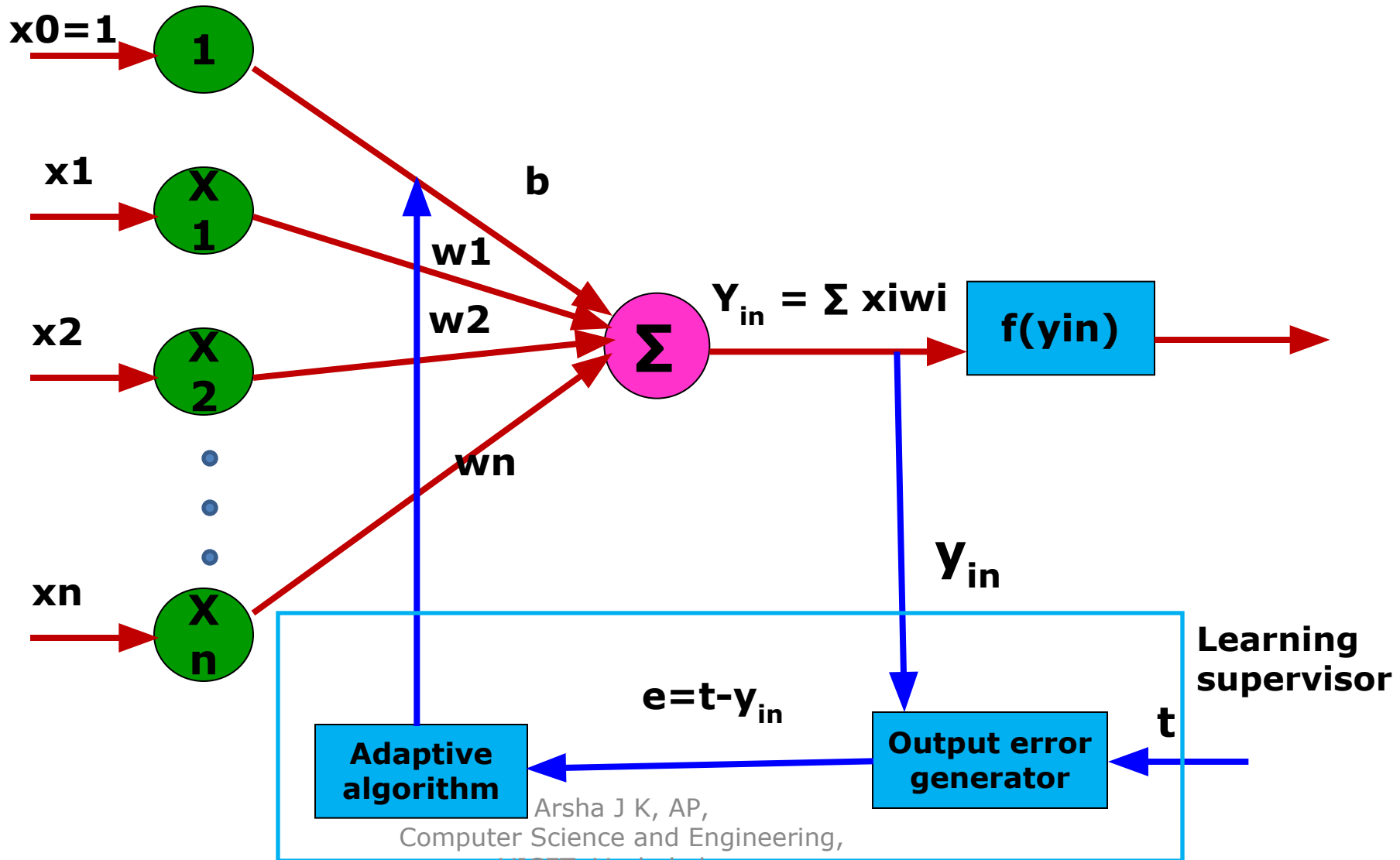
- ✓ Similar to Perceptron rule
- ✓ Perceptron rule is originated from [Hebbian Assumption](#)
- ✓ Delta rule is from [Gradient Descent method](#)
- ✓ Perceptron rule stops after a finite number of learning steps
- ✓ Delta rule continues forever, converging only asymptotically to the solution
- ✓ Delta rule update the weight between the connection so as to minimize the difference between the net input to the output unit and the target value
- ✓ Major aim is to minimize the error over all training patterns

Perceptron	Delta
Originates from hebbian assumption	Derived from gradient-descent method
Stops after a finite number of learning steps	Continuous forever converging asymptotically to the solution
	Minimizes error over all training patterns

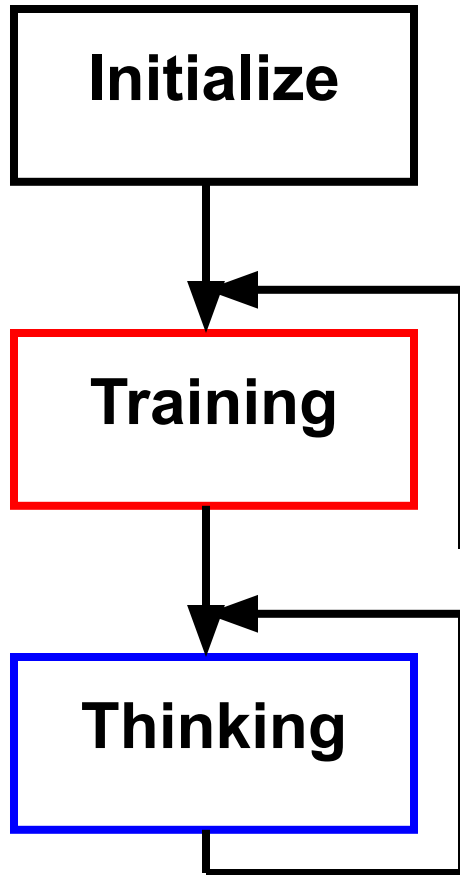
ARCHITECTURE

- ✓ Adaline is a single unit neuron, which receives the input from several units and also from units called bias
- ✓ Basic Adaline model consist of trainable weights
- ✓ Input are either of 2 values(+1 or -1) and the weight have signs(positive or negative)
- ✓ Initially random weights are assigned
- ✓ The net input is calculated is applied to a quantizer transfer function that restore the output to +1 or -1
- ✓ Adaline model compare the actual output with the target output and on the basis of training algorithm, the weights are adjusted

ARCHITECTURE



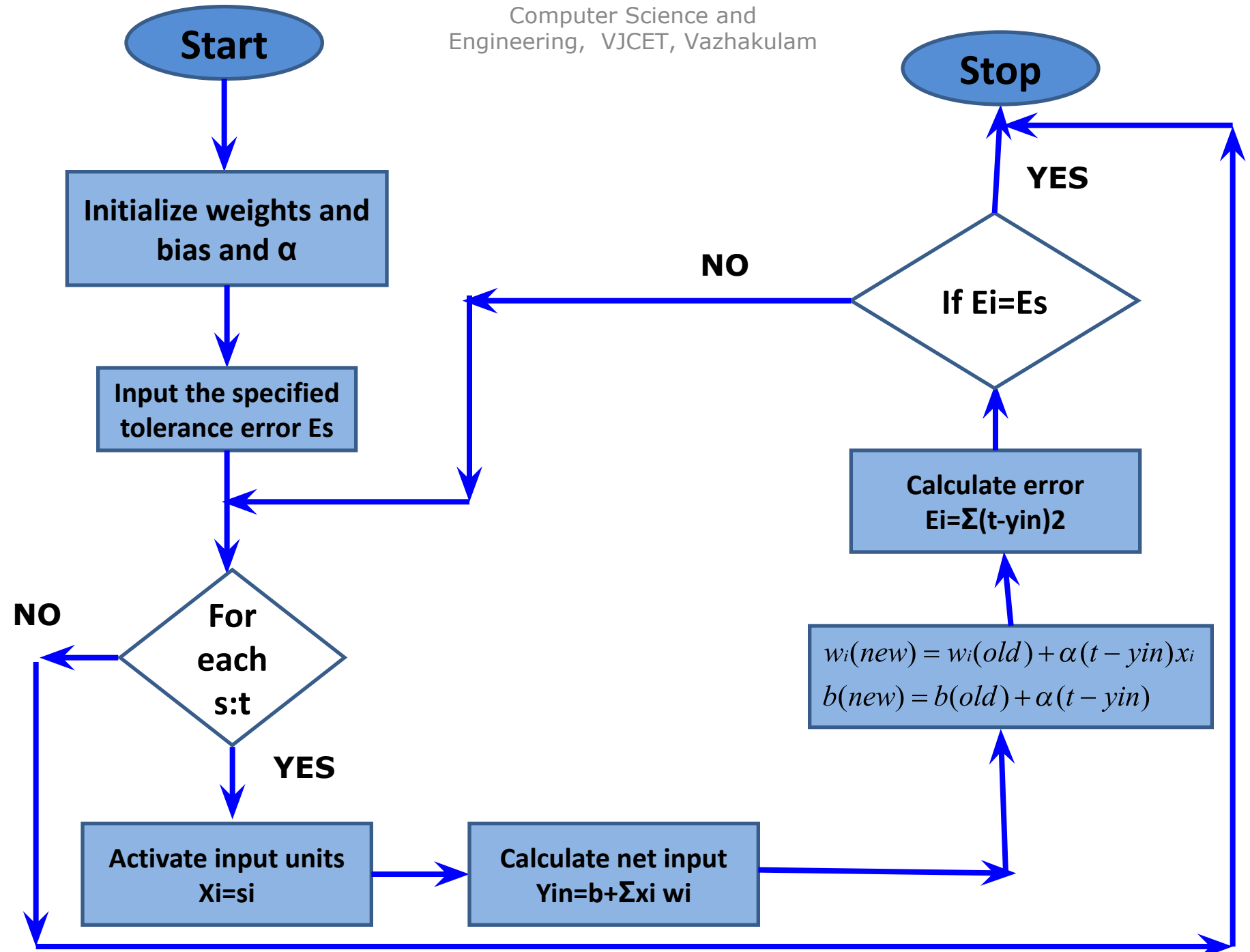
USING ADALINE NETWORKS



- Initialize
 - Assign random weights to all links
- Training
 - Feed-in known inputs in random sequence
 - Simulate the network
 - Compute error between the input and the output (Error Function)
 - Adjust weights (Learning Function)
 - Repeat until total error $< \epsilon$
- Thinking
 - Simulate the network
 - Network will respond to any input
 - Does not guarantee a correct solution even for trained inputs

TRAINING ALGORITHM

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam



ADALINE TRAINING ALGORITHM

Step 0: Weights and bias are set to some random values other than zero. Learning rate α is set

Step 1: Perform Steps 2-6 when stopping condition is false.

Step 2: Perform steps 3-5 for each bipolar training pair $s:t$

Step 3: Set activations for input units $i=1$ to n $x_i=s_i$

Step 4: Calculate the net input to the output unit

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

Step 5: Update the weights and bias for $i=1$ to n

$$w_i(new) = w_i(old) + \alpha(t - y_{in})x_i$$

$$b(new) = b(old) + \alpha(t - y_{in})$$

Step 6: If highest weight change that occurred during training is smaller than a specified tolerance then stop the training else continue. (Stopping condition)

TESTING ALGORITHM

- Step 0: Initialize the weights(from training algo)
- Step 1: Perform steps2-4 for each bipolar input vector x
- Step 2: Set the activations of the input units to x
- Step 3: Calculate the net input

$$y_{in} = b + \sum x_i w_i$$

- Step 4: Apply the activation function over the net input calculated

y =


$$\begin{bmatrix} 1 & \text{If } y_{in} \geq 0 \\ -1 & \text{If } y_{in} < 0 \end{bmatrix}$$

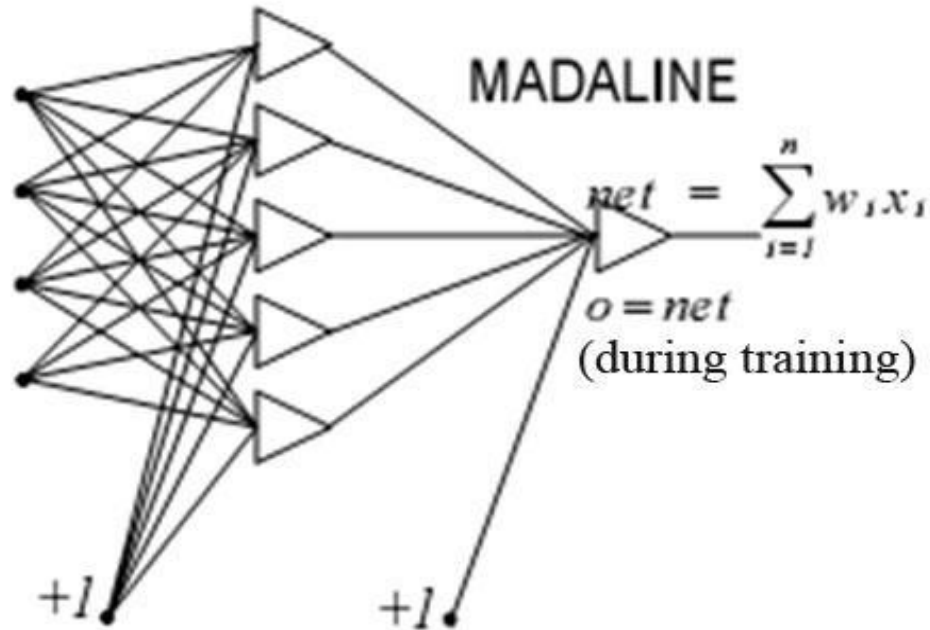
TUTORIAL QUESTIONS

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam

1. Use ADALINE network to train ANDNOT function with bipolar input and targets. Perform 2 epoch of training
2. Implement AND function using ADALINE
3. Using Delta rule find the weight required to perform following classifications: Vectors(1,1,-1,-1) and (-1,-1,-1,-1) belongs to the class having target value 1: vectors (1,1,1,1) and (-1,-1,1,-1) re not belong to the class having target value -1. Use learning rate of 0.5 and assume random values weight. Also test the network

MADALINE NETWORK

MADALINE is a Multilayer Adaptive Linear Element. MADALINE was the first neural network to be applied to a real world problem. It is used in several adaptive filtering process.

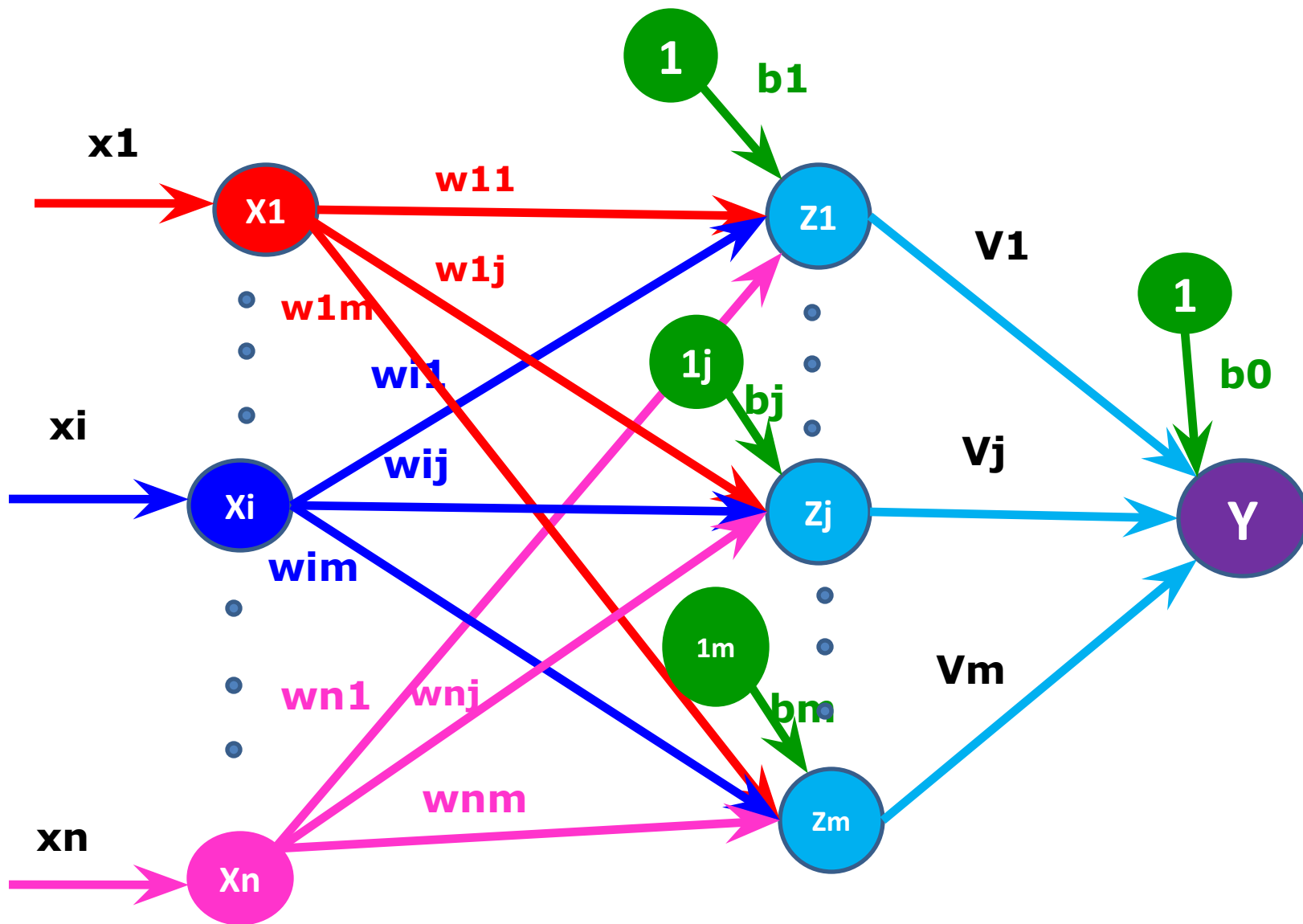


MADALINE

- Two or more adaline are integrated to develop madaline model
- Used for nonlinearly separable logic functions (X-OR) function
- Used for adaptive noise cancellation and adaptive inverse control
- In noise cancellation the objective is to filter out an interference component by identifying a linear model of a measurable noise source and the corresponding immeasurable interference.
- ECG, echo elimination from long distance telephone transmission lines

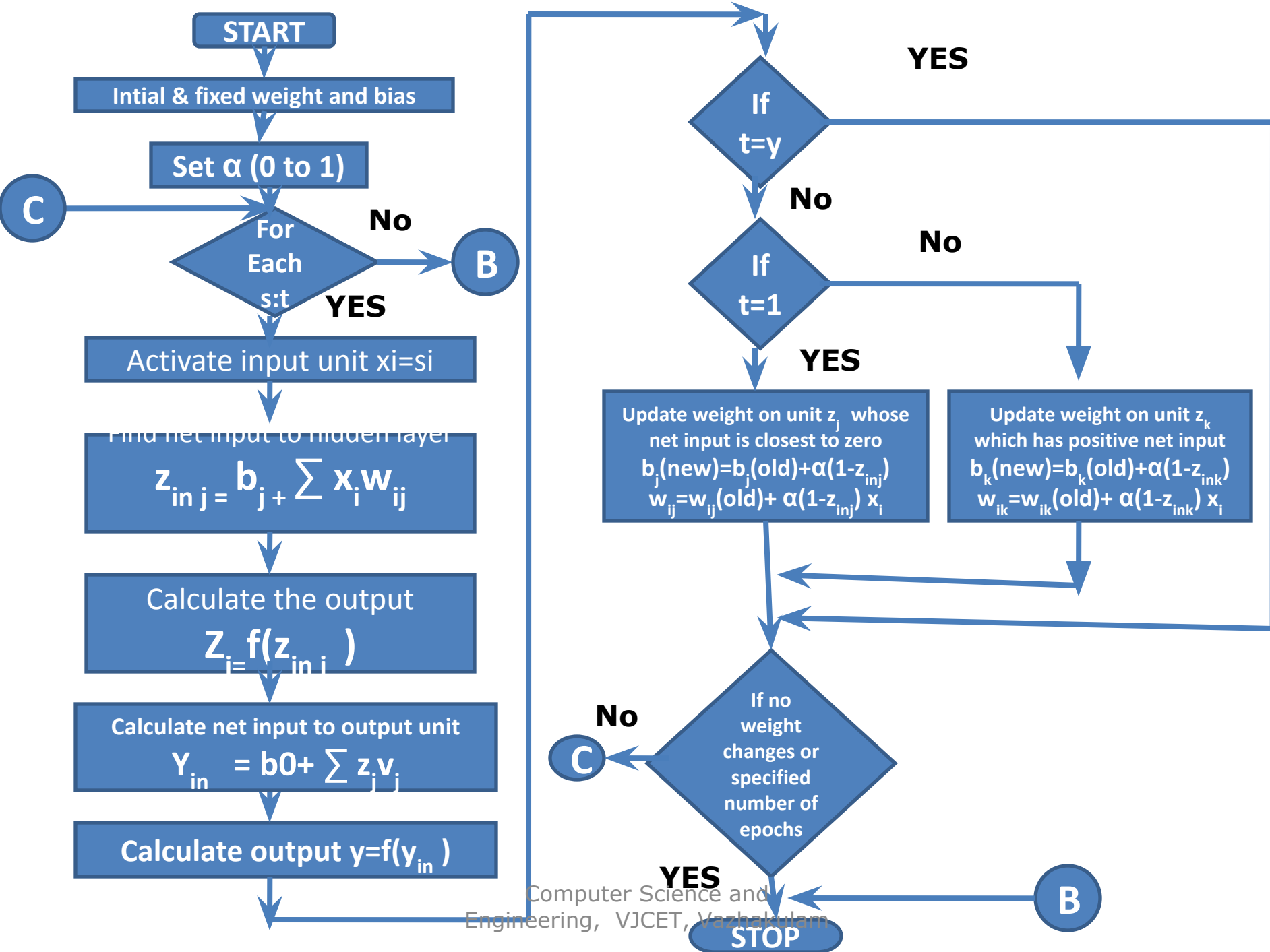
ARCHITECTURE

- ✓ Simple madaline architecture consist of 'n' unit of input layer 'k' unit of Adaline layer and one unit of madaline layer
- ✓ Each neuron in the adaline and madaline layer has a bias of excitation 1
- ✓ Adaline layer is present between the input layer and madaline layer hence the adaline layer can be considered as hidden layer
- ✓ The use of hidden layer gives the net computational capability which is not found in single layer net
- ✓ But this complicate the training process



TRAINING ALGORITHM

Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam



- ✓ Only the weight between the hidden layer and input layer are adjusted
- ✓ Weight for the output layer is fixed
- ✓ The weight $v_1, v_2, v_3 \dots v_j$ and bias b_0 that enter into the output unit Y are delimited so that the response of unit Y is 1
- ✓ Thus the weight entering the Y unit may be taken as

$$v_1 = v_2 = \dots v_j = 1/2$$

- ✓ Bias can be taken as

$$b_0 = 1/2$$

- ✓ The activation function of adaline and madaline unit are

$$f(x) = \begin{cases} 1 & \text{If } x \geq 0 \\ -1 & \text{If } x < 0 \end{cases}$$

- **Step 0:** Initialize the weight. The weight entering the output unit are set. Set initial small random values for adaline weights. Also set initial learning rate α
- **Step 1 :** When stopping condition is false, perform step 2-3
- **Step 2:** For each bipolar training pair s:t, perform 3-7
- **Step 3:** Activate input layer unit. For $i=1$ to n $x_i=s_i$
- **Step 4:** Calculate the net input unit to each hidden adaline unit

$$z_{in\ j} = b_j + \sum_{i=1}^n x_i w_{ij} \quad , j= 1 \text{ to } m$$

- **Step 5:** Find output of each hidden unit

$$z_j = f(z_{in\ j})$$

- **Step 6:** Find output of net

$$y_{in} = b_0 + \sum_{j=1}^m z_j v_j$$

$$y = f(y_{in})$$

- **Step 7:** Calculate the error and update the weight

- A: if $t=y$, no weight updation is required
- B: if $t \neq y$, & $t = +1$ update weight on z_j where net input closer to zero

$$b_j(\text{new}) = b_j(\text{old}) + \alpha(1 - z_{inj})$$

$$w_{ij} = w_{ij}(\text{old}) + \alpha(1 - z_{inj}) x_i$$

- **C: if $t \neq y$ and $t = -1$ update the weight on unit z_k whose net input is positive**

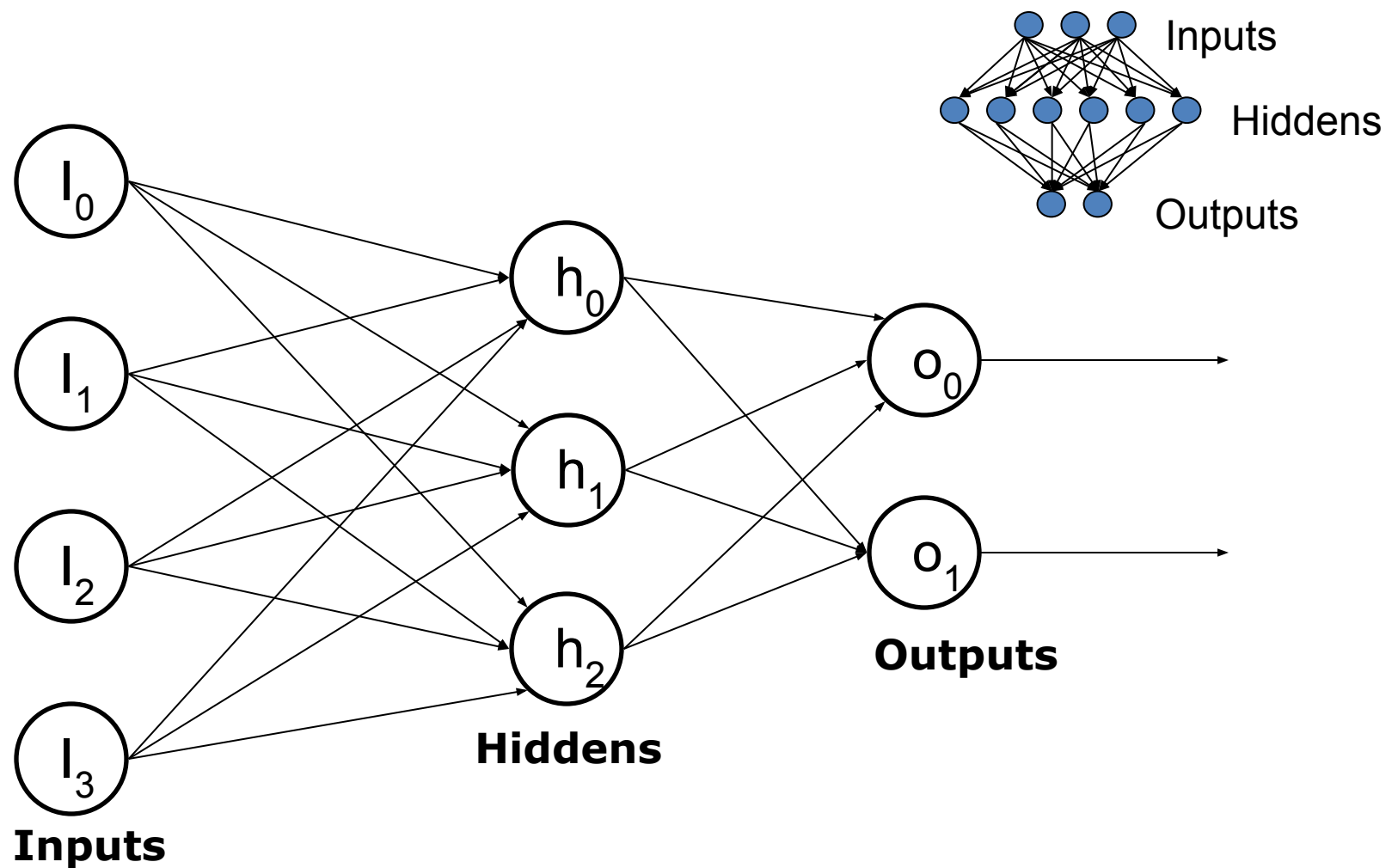
$$b_k(\text{new}) = b_k(\text{old}) + \alpha(1 - z_{ink})$$

$$w_{ik} = w_{ik}(\text{old}) + \alpha(1 - z_{ink}) x_i$$

- **Step 8:** Test for stopping condition (if there is no weight change or weight reaches the satisfactory level, or if a specified maximum number of weight updation have been performed then stop or else continue

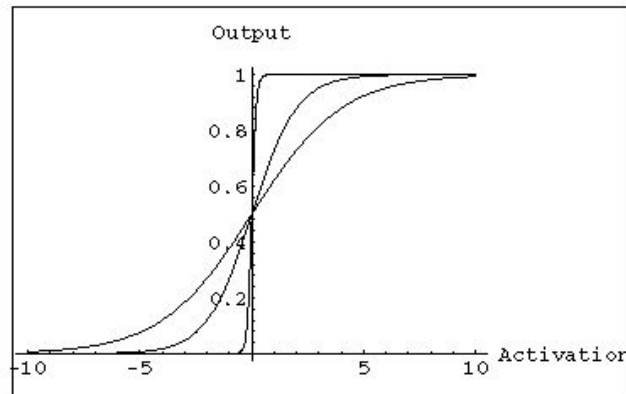
- A training procedure which allows multilayer feed forward Neural Networks to be trained.
- Can theoretically perform “any” input-output mapping.
- Can learn to solve linearly inseparable problems.

MULTILAYER FEEDFORWARD NETWORK



MULTILAYER FEEDFORWARD NETWORK: ACTIVATION AND TRAINING

- For feed forward networks:
 - A continuous function can be
 - differentiated allowing
 - gradient-descent.
 - Back propagation is an example of a gradient-descent technique.
 - Uses sigmoid (binary or bipolar) activation function.



In multilayer networks, the activation function is usually more complex than just a threshold function, like $1/[1+\exp(-x)]$ or even $2/[1+\exp(-x)] - 1$ to allow for inhibition, etc.

GRADIENT DESCENT

- Gradient-Descent(training_examples, η)
- Each training example is a pair of the form $\langle (x_1, \dots, x_n), t \rangle$ where $(x_1, \dots, x_n)$ is the vector of input values, and t is the target output value, η is the learning rate (e.g. 0.1)
- Initialize each w_i to some small random value
- Until the termination condition is met, Do
 - Initialize each Δw_i to zero
 - For each $\langle (x_1, \dots, x_n), t \rangle$ in training_examples Do

- ✓ Input the instance ($x_1, \dots, x_n$) to the linear unit and compute the output o
- ✓ For each linear unit weight w_i Do

- $\Delta w_i = \Delta w_i + \eta (t - o) x_i$
- For each linear unit weight w_i Do
- $w_i = w_i + \Delta w_i$

MODES OF GRADIENT DESCENT

- Batch mode : gradient descent

$w = w - \eta \nabla E_D[w]$ over the entire data D

$$E_D[w] = \frac{1}{2} \sum_d (t_d - o_d)^2$$

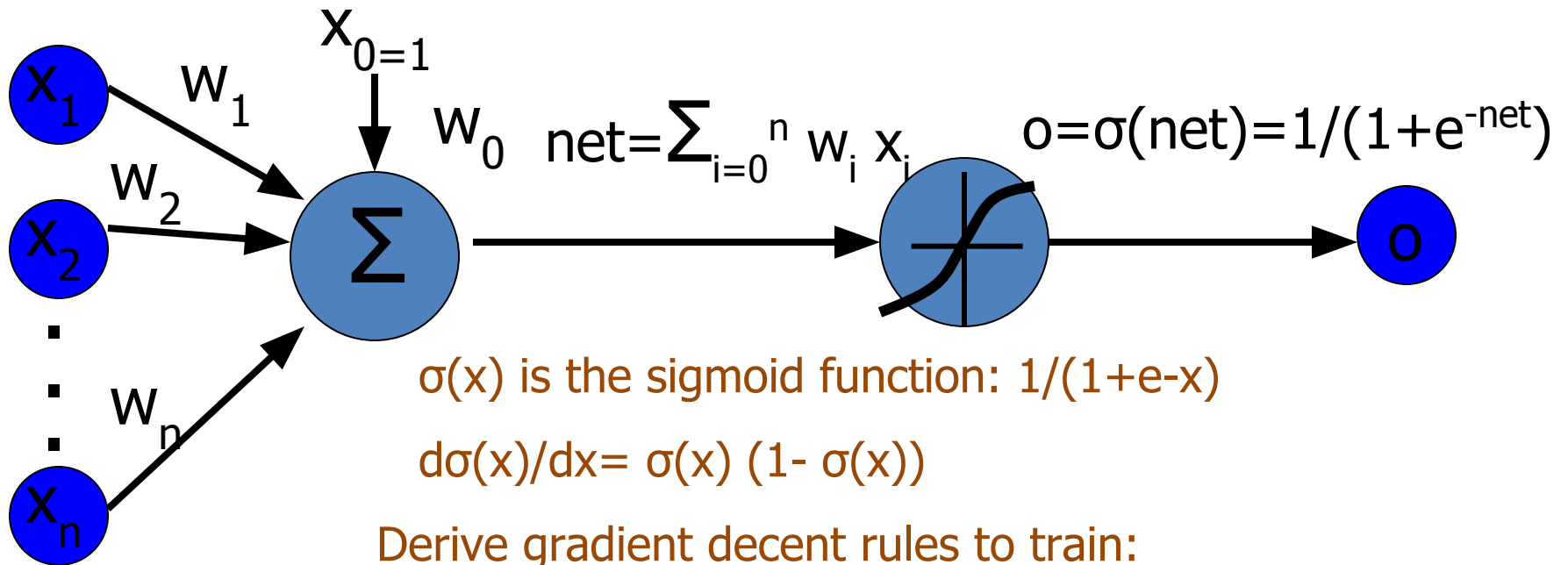
- Incremental mode: gradient descent

$w = w - \eta \nabla E_d[w]$ over individual training examples d

$$E_d[w] = \frac{1}{2} (t_d - o_d)^2$$

- Incremental Gradient Descent can approximate Batch Gradient Descent arbitrarily closely if η is small enough.

SIGMOID ACTIVATION FUNCTION



$\sigma(x)$ is the sigmoid function: $1/(1+e^{-x})$

$$d\sigma(x)/dx = \sigma(x) (1 - \sigma(x))$$

Derive gradient decent rules to train:

- one sigmoid function
$$\partial E / \partial w_i = -\sum d(t_d - o_d) o_d (1 - o_d) x_i$$
- Multilayer networks of sigmoid units
backpropagation

BACK PROPAGATION NETWORK

- Backprop implements a gradient descent search through the space of possible network weights, iteratively reducing the error E , between training example target values and the network outputs.
- Guaranteed to converge only towards some local minima.

BACKPROPAGATION

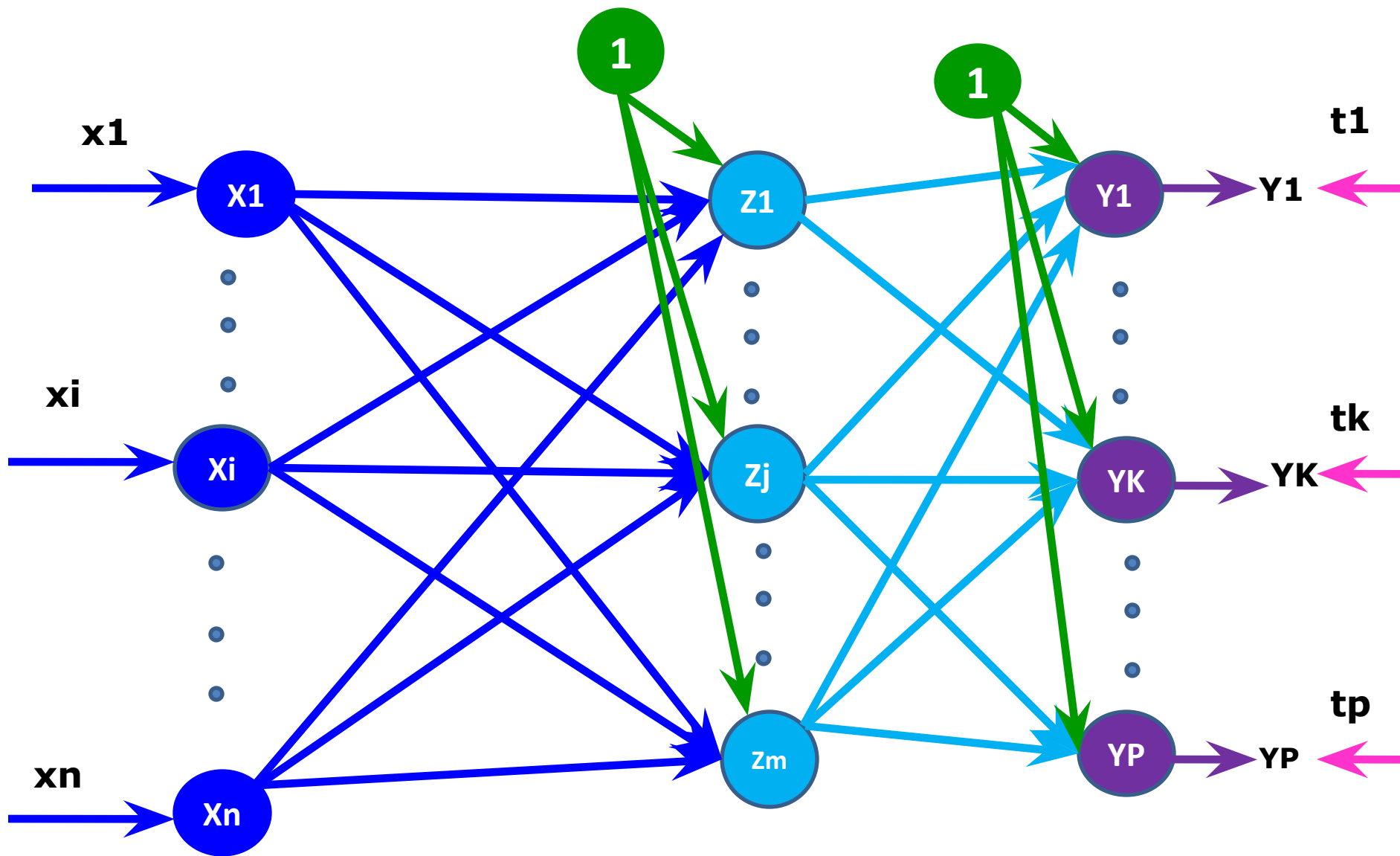
- Gradient descent over entire network weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum -in practice often works well (can be invoked multiple times with different initial weights)
- Often include weight momentum term
$$\Delta w_{i,j}(t) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(t-1)$$
- Minimizes error training examples
- Will it generalize well to unseen instances (over-fitting)?
- Training can be slow typical 1000-10000 iterations (use Levenberg-Marquardt instead of gradient descent)

- ✓ Back propagation is a training method used for a multi layer feed forward network.
- ✓ The network associated with Back propagation learning algorithm is called BACK PROPAGATION NETWORK
- ✓ Processing elements with continuous differentiable activation functions
- ✓ It is also called the **generalized delta rule**.
- ✓ It is a gradient descent method which minimizes the total squared error of the output computed by the net.
- ✓ Any neural network is expected to respond correctly to the input patterns that are used for training which is termed as memorization and it should respond reasonably to input that is similar to but not the same as the samples used for training which is called generalization.

- ✓ The training of a neural network by back propagation takes place in three stages
 - ✓ 1. Feed forward of the input pattern
 - ✓ 2. Calculation and Back propagation of the associated error
 - ✓ 3. Adjustments of the weights
- ✓ After the neural network is trained, the neural network has to compute the feed forward phase only.
- ✓ Even if the training is slow, the trained net can produce its output immediately.

ARCHITECTURE

- ✓ The output units and the hidden units can have biases.
- ✓ These bias terms are like weights on connections from units whose output is always 1.
- ✓ During feed forward the signals flow in the forward direction i.e. from input unit to hidden unit and finally to the output unit.
- ✓ During back propagation phase of learning, the signals flow in the reverse direction.

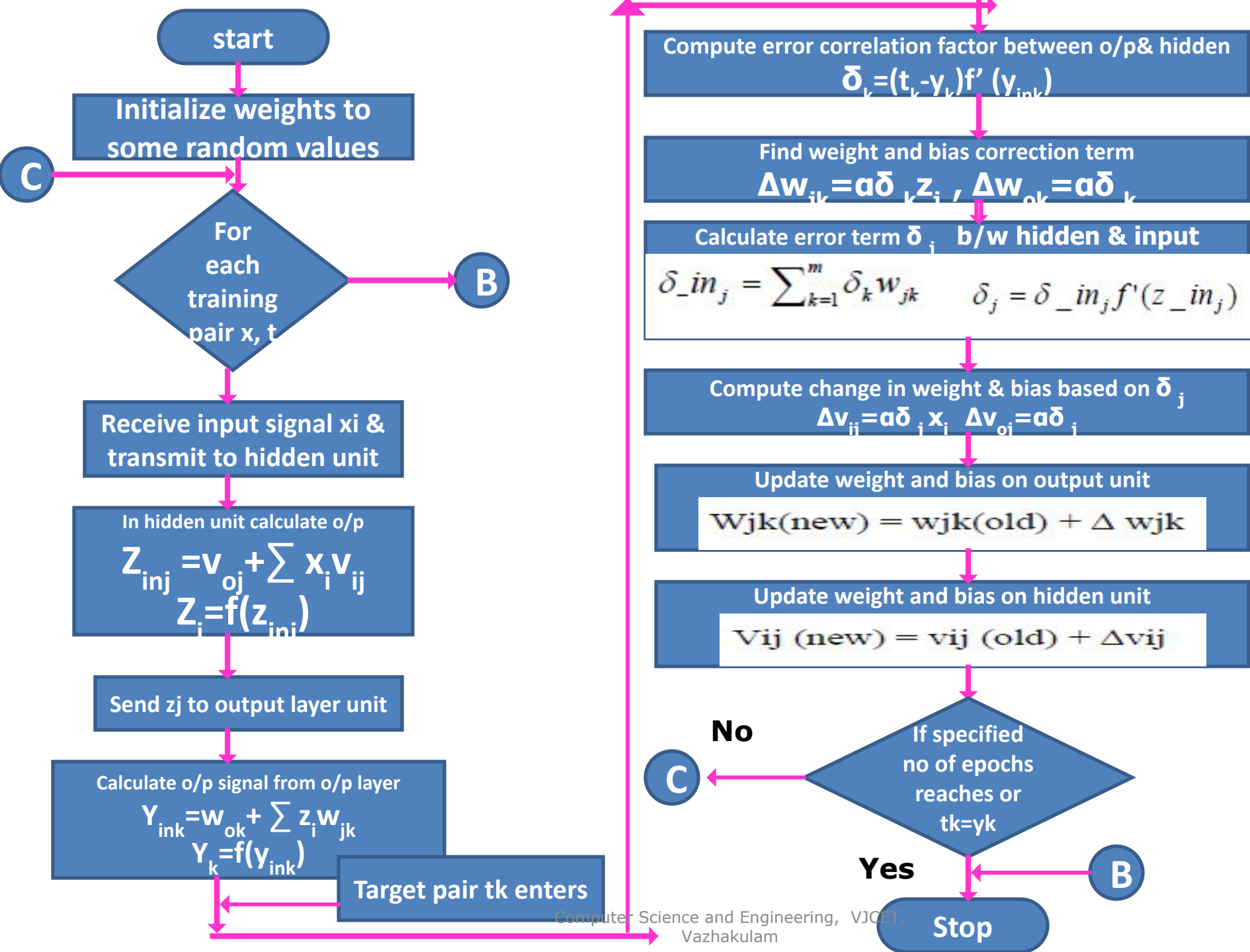


ALGORITHM

- ✓ The training involves three stages
 - ✓ 1. Feed forward of the input training pattern
 - ✓ 2. Back propagation of the associated error
 - ✓ 3. Adjustments of the weights.
- ✓ During feed forward, each input unit (X_i) receives an input signal and sends this signal to each of the hidden units $Z_1, Z_2, \dots, Z_n$.
- ✓ Each hidden unit computes its activation and sends its signal to each output unit.
- ✓ Each output unit computes its activation to compute the output or the response of the neural net for the given input pattern.

- ✓ During training, each output unit compares its computed activation y_k , with its target value t_k to determine the associated error for the particular pattern.
- ✓ Based on this error the factor δ_k for all m values are computed.
- ✓ This computed δ_k is used to propagate the error at the output unit Y_k back to all units in the hidden layer.
- ✓ At a later stage it is also used for updation of weights between the output and the hidden layer.
- ✓ In the same way δ_j for all p values are computed for each hidden unit Z_j .
- ✓ The values of δ_j are not sent back to the input units but are used to update the weights between the hidden layer and the input layer.

- ✓ Once all the ∂ factors are known, the weights for all layers are changed simultaneously.
- ✓ The adjustment to all weights w_{jk} is based on the factor ∂_k and the activation z_j of the hidden unit Z_j .
- ✓ The change in weight to the connection between the input layer and the hidden layer is based on ∂_j and the activation x_i of the input unit



ACTIVATION FUNCTION

- ✓ An activation function for a back propagation net should have important characteristics.
- ✓ It should be continuous, Differentiable and monotonically non-decreasing.
- ✓ For computational efficiency, it is better if the derivative is easy to calculate.
- ✓ For the commonly used activation function, the derivative can be expressed in terms of the value of the function itself. The function is expected to saturate asymptotically.
- ✓ The commonly used activation function is the binary sigmoidal function.

TRAINING ALGORITHM

- ✓ The activation function used for a back propagation neural network can be either a bipolar sigmoid or a binary sigmoid.
- ✓ The form of data plays an important role in choosing the type of the activation function.
- ✓ Because of the relationship between the value of the function and its derivative, additional evaluations of exponential functions are not required to be computed.

- **Step 0:** Initialize weights
- **Step 1:** While stopping condition is false, do steps 2 to 9
- **Step 2:** For each training pair, do steps 3 - 8

Feed forward

Step 3: Input unit receives input signal and propagates it to all units in the hidden layer

Step 4: Each hidden unit sums its weighted input signals

Step 5: Each output unit sums its weighted input signals and applied its activation function to compute its output signal.

Back propagation

Step 6: Each output unit receives a target pattern corresponding to the input training pattern, computes its error information term

$$\delta_k = (t_k - y_k) f'(y_{\text{ink}})$$

Calculates its bias correction term

$$\Delta W_{ok} = \alpha \delta_k$$

And sends δ_k to units in the layer below

Step 7: Each hidden unit sums its delta inputs

$$\delta_{\text{in}_j} = \sum_{k=1}^m \delta_k w_{jk}$$

Multiplies by the derivative of its activation function to calculate its error information term

$$\delta_j = \delta_{\text{in}_j} f'(z_{\text{in}_j})$$

Calculates its weight correction term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

And calculates its bias correction term

$$\Delta v_{oj} = \alpha \delta_j$$

Update weights and biases

Step 8: Each output unit updates its bias and weights

$$W_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

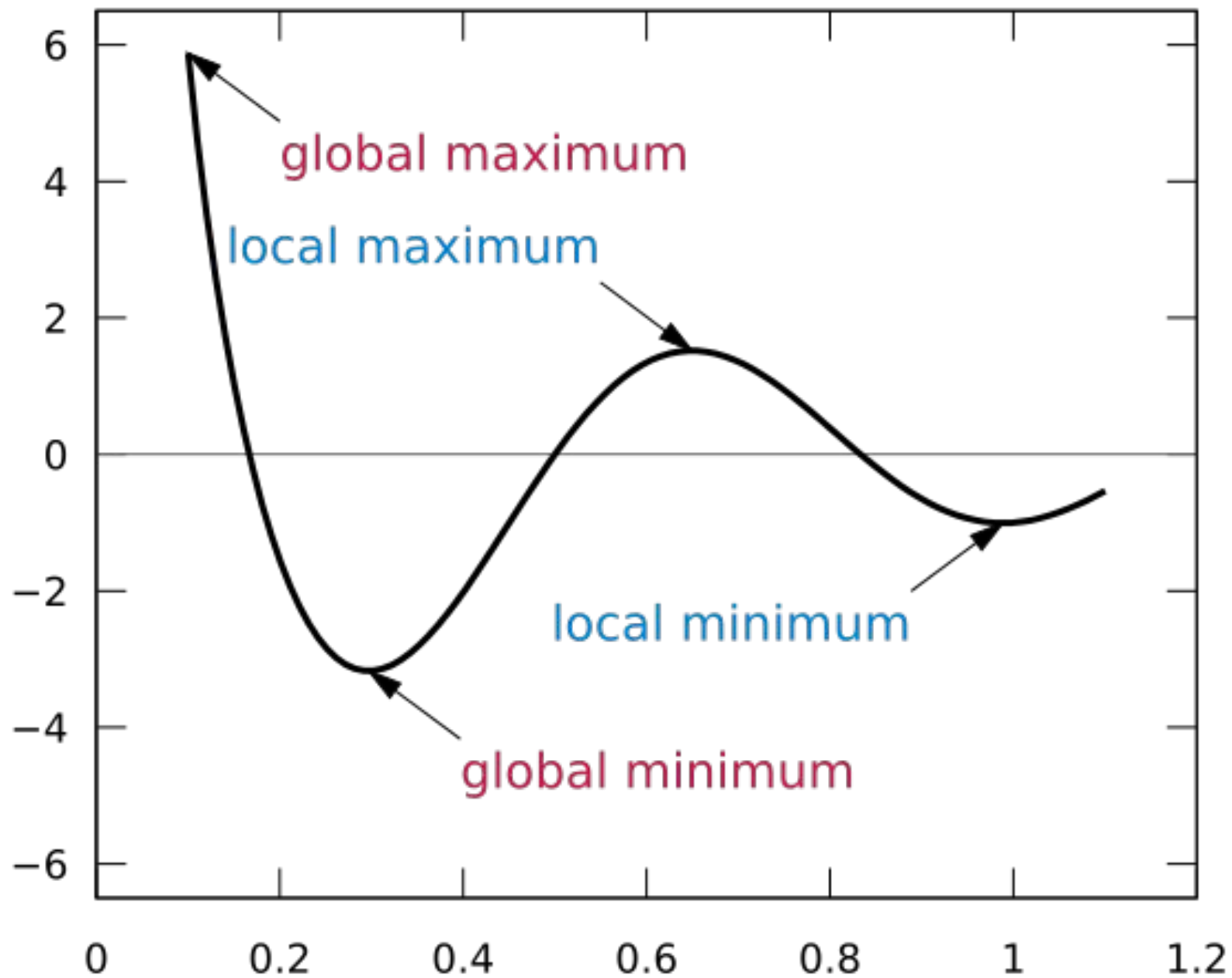
Each hidden unit updates its bias and weights

$$V_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}$$

Step 9: Test stopping condition. The stopping condition may be certain number of epochs reached or when the actual output equal to target value

- ✓ The above algo uses the incremental approach for updation of weight
- ✓ That is weights are being changed immediately after a training pattern is present
- ✓ There is another way of training called batch mode training where the weights are changed only after all the training pattern are presented
- ✓ Batch mode requires additional local storage for each connection to maintain the immediate weight change
- ✓ When a BPN used as a classifier, it is equivalent to the *optimal Bayesian Discriminator* function for asymptotically large set of training patterns

- ✓ If the BPN algorithm converges at all , then it may get stuck with local minima and may be unable to find a satisfactory solution
- ✓ The randomness of the algorithm helps it to get out of local minima
- ✓ The error function may have large number of global minima because of permutation of weight that keep the network input output function unchanged



LEARNING FACTORS OF BACK PROPOGATION NETWORK

- **Training of BPN is based on the choice of various parameters**
- **Convergence of BPN is based on some important learning factors such as**
 - Initial weight
 - The learning rate
 - The updation rule
 - Size and nature of training set
 - Architecture (number of layers and number of neurons in each layer)

INITIAL WEIGHT

- The weights are initialized with some random values
- The choice of initial weight is determines how fast the network converges
- The initial weight cannot be very high becoz sigmoid activation function used here may get saturated from the beginning itself and the system may be stuck at the local minima
- One method for choosing weight is in the range

$$\left[-\frac{3}{\sqrt{o_i}}, \frac{3}{\sqrt{o_i}} \right]$$

- Where o_i is the number of processing elements j that feed forward to processing element i

- The initialization can also be done by a method called **Nguyen- Widrow Initialization**
- Leads to faster convergence of network
- Improves the learning ability of hidden layer
- The random initialization of weight connecting input neuron to the hidden neuron is obtained by

$$V_{ij}(\text{new}) = \gamma \cdot v_{ij}(\text{old}) / ||v_j(\text{old})||$$

LEARNING RATE α

- Affect the convergence of BPN
- A large value of alpha may speedup the convergence but may result in overshooting
- The range of alpha from 10^{-3} to 10 has been used successfully for several backpropagation algorithms experiments
- Slower learning rate lead to slower learning

Momentum factor

- The gradient descent is very slow if the learning rate is small and oscillate widely if alpha is too large
- One method that allows a large learning rate without oscillation is by adding a momentum factor to the normal gradient descent method

GENERALIZATION

- A network is said to be generalized when it sensibly interpolate with input network that are new to the network
- When there are many trainable parameters for a given amount of training data, the network learn well but does not generalize well
- This is usually called over fitting or over training
- One solution is to monitor error on the test set and terminate the training when error increases
- With a small number of trainable parameters, the network fail to learn training data set to test the data set
- However computationally large number of nodes is capable of memorizing the training set at the cost of generalization
- As a result smaller net are preferred than larger ones

Number of Training Data

- Training data should be sufficient and proper
- There exist a rule of thumb which state that the training data should cover the entire expected input space, and while training , the training vector pair should be selected randomly from the set

Number of Hidden Layer nodes

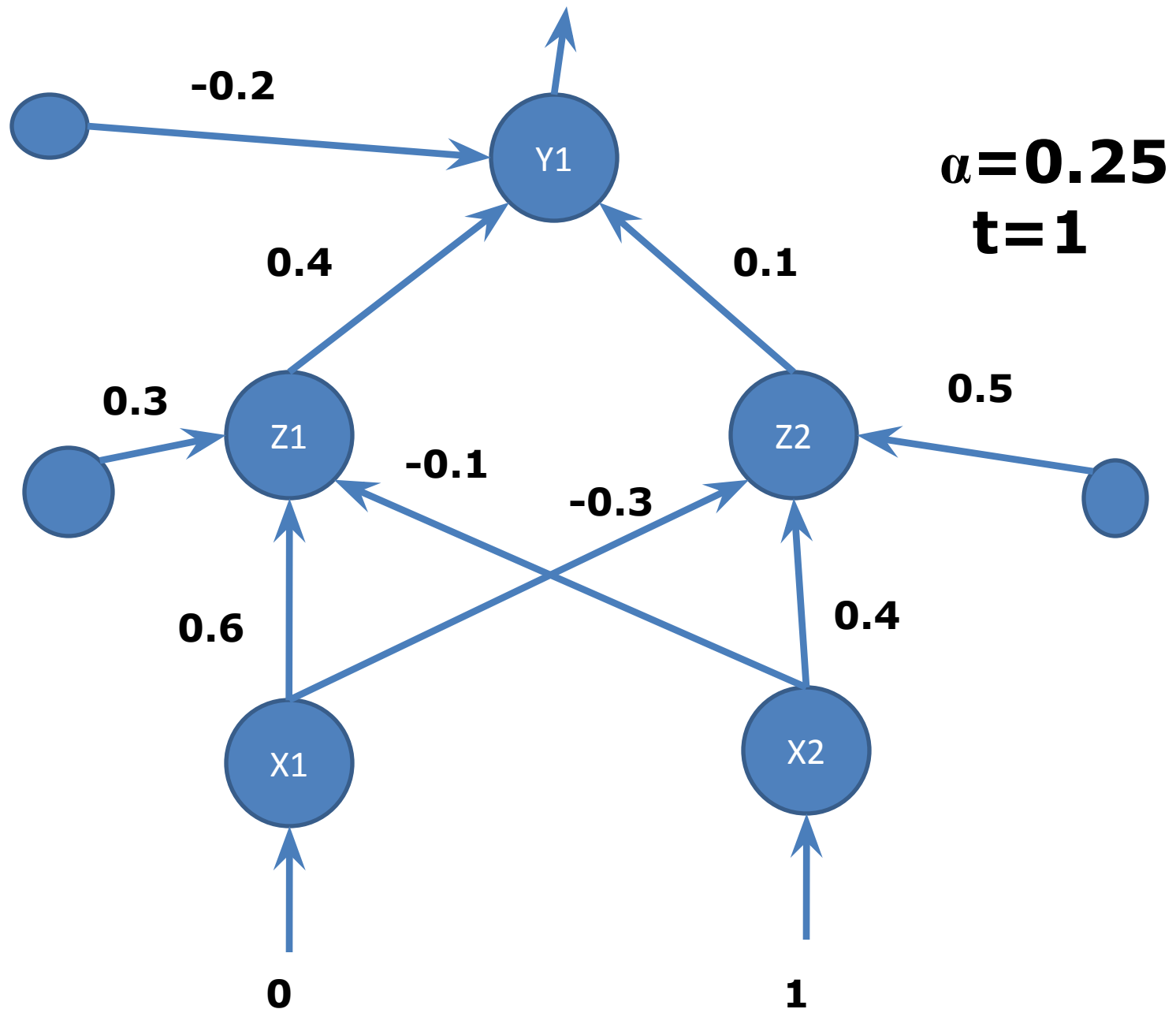
- If there exist more than one hidden layer in BPN, then the calculation performed for a single layer are repeated for all the layers and summed up at the end
- In case of all multilayer feed forward network, the size of hidden layer is very important
- The number of hidden layer need for an application determined separately
- The size of hidden layer is determined experimentally
- For a network of reasonable size, the size of hidden layer has only be a relatively small fraction of input layer
- For example if the network does not converges to a solution , it may need more hidden layers

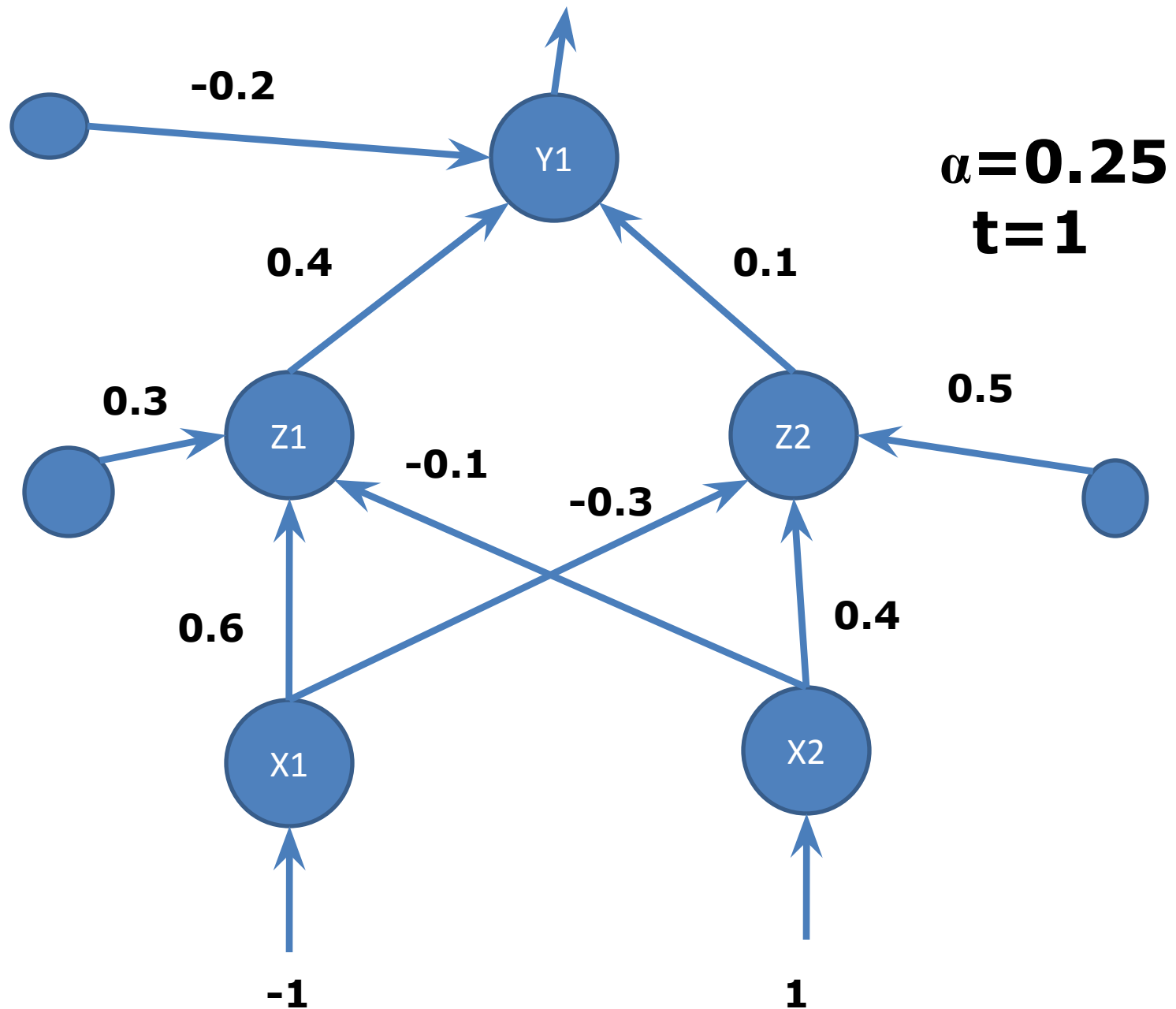
Testing Algorithm

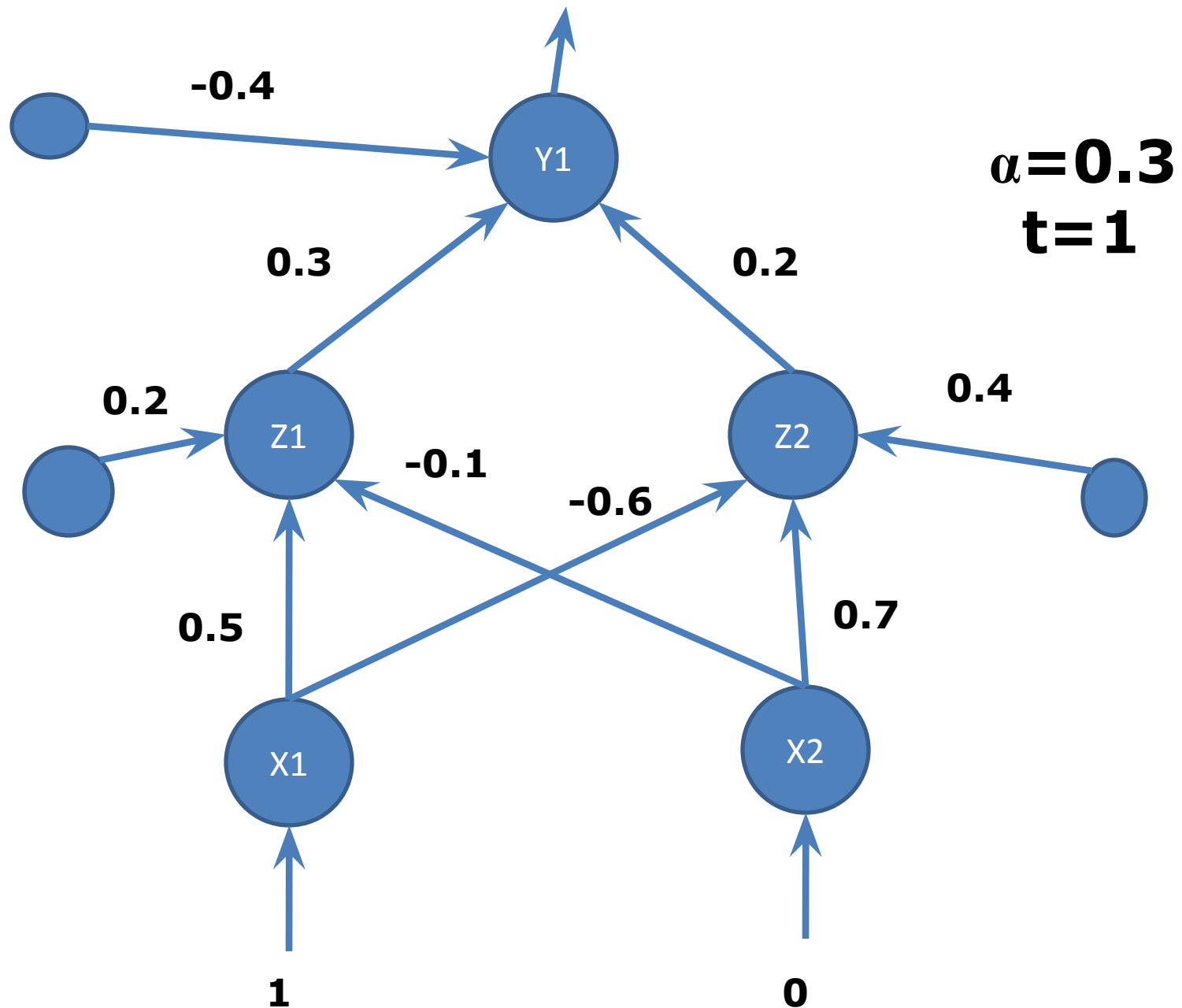
- Step 0: Initialize weight. The weight are taken from training algorithm
- Step 1: Perform step 2- 4 for each input vector
- Step 2: Set the activation of input unit for x_i
- Step 3: Calculate the net input to hidden unit and its output
- Step 4: Now compute the output for output layer unit. Use sigmoidal activation function

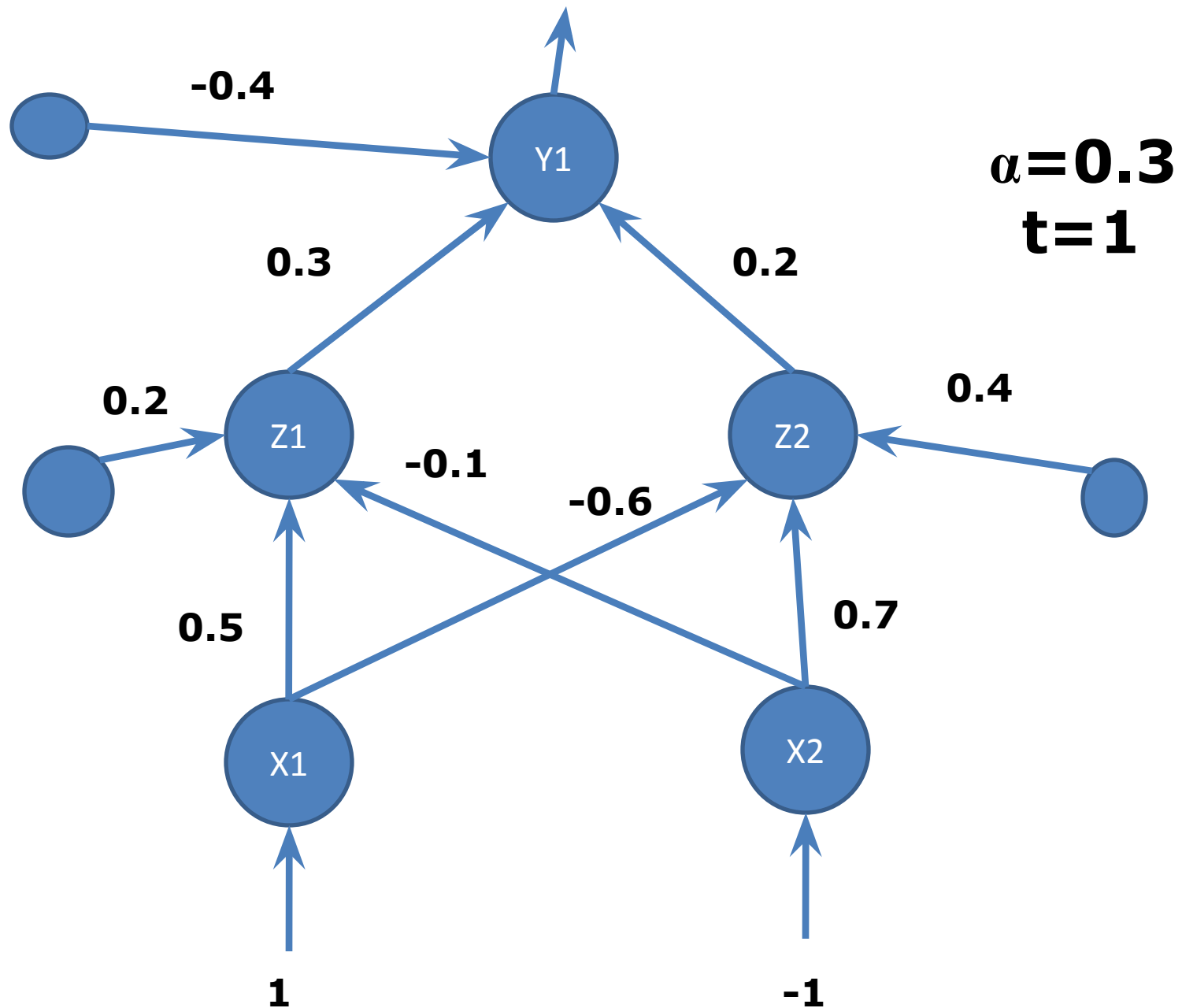
PROBLEMS

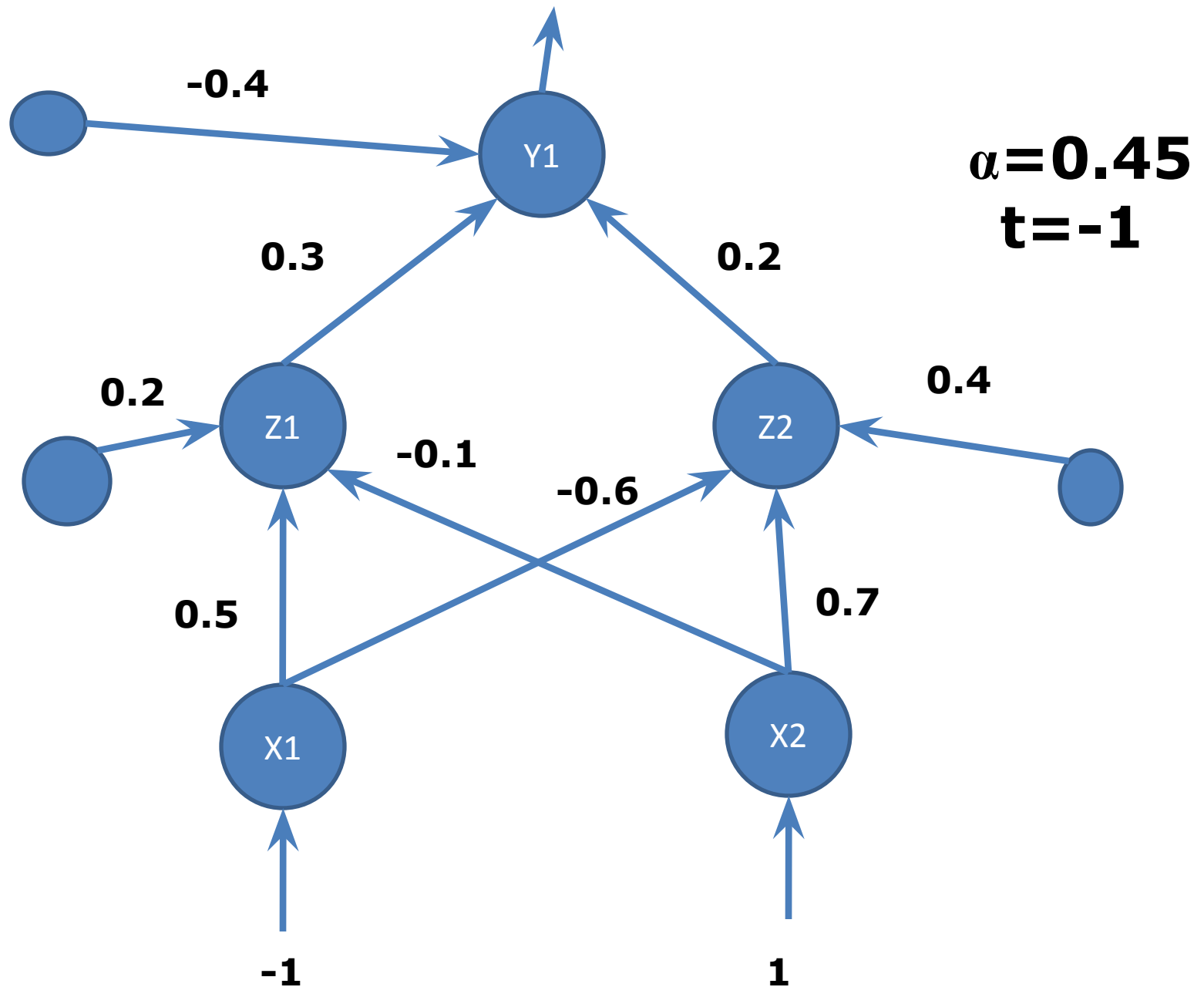
Arsha J K, AP,
Computer Science and Engineering,
VJCET, Vazhakulam











APPLICATIONS OF BACKPROPAGATION NETWORK

- ❑ Load forecasting problems in power systems.
- ❑ Image processing.
- ❑ Fault diagnosis and fault detection.
- ❑ Gesture recognition, speech recognition.
- ❑ Signature verification.
- ❑ Bioinformatics.
- ❑ Structural engineering design (civil).

Tutorial Questions

1. Find the weights required to perform classification of patterns shown below using perceptron network. The patterns $(1,1,-1)$ and $(1,-1,-1)$ are belonging to the target class -1. The patterns $(-1,1,1)$ and $(-1,-1,1)$ are belonging to the target class +1. Assume suitable learning rate and initial weights.
2. Explain the architecture and training algorithm of Adaline network . Use Adaline network to train NOR logic function with bipolar inputs and targets. Perform 2 epochs of training.

Find the weight required to perform the following classification using perceptron network. The vectors $(1,1,-1,-1)$ and $(1,-1,1,-1)$ are belongs to the class so have the target value 1, vectors $(-1,-1,-1,1)$ and $(-1,-1,1,1)$ are not belongs to the class so have the target value -1. Assume learning rate 1 and initial weights as 0

Implement AND function using ADALINE network

Using Delta rule find the weight required to perform following classifications: Vectors $(1,1,-1,-1)$ and $(-1,-1,-1,-1)$ belongs to the class having target value 1: vectors $(1,1,1,1)$ and $(-1,-1,1,-1)$ are not belong to the class having target value -1. Use learning rate of 0.5 and assume random values weight. Also test the network

Using Back Propagation network find the new weights for the network. It is presented with the input pattern $[1,0]$ $[1,-1]$ $-1,1]$ and target output is 1. Use Learning rate 0.3